

UNIVERSIDAD DEL VALLE DE GUATEMALA

Data Science

Facultad de Ingeniería

Sección 40

Erick Marroquín



Laboratorio 5. Minería de Textos

Joel Jaquez - 23369

Fernando Ruiz - 23065

Kevin Villagran - 23584

GUATEMALA, 30 de agosto 2026

Enlace al repositorio:

<https://github.com/DerNait/data-science-lab-5-mineria-de-textos>

 Informe Final - Laboratorio 5. Minería de Textos

Introducción

Este laboratorio trabaja el conjunto Natural Language Processing with Disaster Tweets de Kaggle, donde cada tweet viene etiquetado a mano según hable de un desastre real o no. El archivo train.csv trae 7,613 tweets. Después de quitar los duplicados exactos y los textos idénticos que traían etiquetas contrarias quedan 7,485, que son los que se usan en todo el trabajo.

Sobre ese corpus se hizo el análisis exploratorio, la limpieza y normalización del texto, el conteo de unigramas, bigramas y trigramas con sus probabilidades, catorce combinaciones de modelo de clasificación, una función que recibe un tweet sin preprocesar y devuelve su clase, el análisis de sentimiento con VADER y el reentrenamiento del modelo con una variable de negatividad.

El modelo que quedó seleccionado es TF-IDF con unigramas y bigramas más regresión logística con pesos balanceados. Alcanza 0.7857 de macro-F1 y 79.23% de exactitud sobre 1,497 tweets de prueba, frente a 0.3646 y 57.38% de un clasificador que siempre responde la clase mayoritaria. La exhaustividad sobre la clase de desastre es 0.7241, o sea que detecta 7 de cada 10 desastres reales.

Hay un resultado que ordena el informe entero, y tiene que ver con la amplitud más que con la cima. Las catorce combinaciones de algoritmo y representación caben todas dentro de 0.0350 de macro-F1, y los seis mejores candidatos quedan dentro de una desviación estándar en validación cruzada. Lo que limita el desempeño es el lenguaje figurado y la ambigüedad de algunas etiquetas del conjunto.

Los datos y su preparación

Origen y contenido

El conjunto viene de la competencia Natural Language Processing with Disaster Tweets de Kaggle. Los tweets fueron recolectados por Figure Eight y cada uno fue etiquetado por anotadores humanos. La descarga se hace con el script src/download_dataset.py, que además verifica la huella SHA256 del archivo para que todo el equipo trabaje sobre exactamente el mismo insumo.

Vale la pena aclarar un punto sobre el tamaño. El enunciado menciona más de 10,500 filas, y eso corresponde a la suma de train.csv y test.csv. El archivo test.csv de la competencia no trae la columna target, porque es el conjunto de evaluación ciega de Kaggle. Entonces todo el análisis se hace sobre train.csv, y la partición de entrenamiento y prueba sale de dividir ese mismo archivo.

La Tabla 1 resume las cinco columnas originales del archivo, con su tipo, sus valores ausentes y cuántos valores distintos toma cada una.

Columna	Tipo	Nulos	% nulos	Valores únicos
id	entero	0	0.00	7,485
keyword	texto	56	0.75	221
location	texto	2,472	33.03	3,321
text	texto	0	0.00	7,485
target	entero	0	0.00	2

Tabla 1. Composición del conjunto después de la limpieza de filas duplicadas.

La columna location tiene un tercio de valores ausentes y es texto libre escrito por el usuario, con 3,321 valores distintos que incluyen cosas como Everywhere o nombres de lugares que no existen. Por eso no se usó como variable del modelo. La columna keyword sí es útil y se analiza más adelante.

Duplicados y etiquetas contradictorias

Antes de partir los datos se revisaron las filas repetidas. Se encontraron 51 duplicados exactos, o sea el mismo texto con la misma etiqueta, y 18 textos idénticos que aparecían con las dos etiquetas a la vez. Esos 18 casos son interesantes por lo que dicen del conjunto, ni siquiera los anotadores humanos se pusieron de acuerdo entre ellos sobre el mismo texto.

En total se eliminan 128 filas y quedan 7,485. Los duplicados se quitaron antes de partir en entrenamiento y prueba, porque si un texto repetido cae en los dos lados el modelo estaría siendo evaluado sobre algo que ya vio, y las métricas saldrían mejores de lo que en realidad son.

Partición de entrenamiento y prueba

La partición es 80% de entrenamiento y 20% de prueba, estratificada por la clase para que las dos partes conserven la proporción original. Quedan 5,988 tweets de entrenamiento y 1,497 de prueba.

Un detalle que resultó importante. La partición se guardó en disco como una lista de identificadores en `data/processed/particion.parquet` y todos los análisis la leen de ahí en lugar de volver a partir los datos. El motivo es que `train_test_split` con la misma semilla devuelve la misma partición solo si recibe las filas en el mismo orden. Basta con que alguien filtre el `DataFrame` o cambie la regla de deduplicación para que la partición se mueva sin que nadie lo note, y entonces el modelo con la variable de negatividad estaría midiéndose contra un conjunto de prueba distinto al del modelo base. Congelarla en disco deja todas las métricas del informe comparables entre sí. La semilla es 42 y está fija en `src/config.py`.

Limpieza y preprocesamiento

Dos limpiezas, porque son dos tareas distintas

El laboratorio pide dos cosas que parecen la misma pero no lo son. La clasificación de desastre contra no desastre es una tarea de tema, y ahí conviene reducir el vocabulario todo lo posible a lo temáticamente relevante. El análisis de sentimiento es una tarea de opinión, y necesita justo lo que la otra descarta, o sea las mayúsculas, los signos expresivos y sobre todo las negaciones.

Por eso el módulo `src/preprocesamiento.py` expone dos funciones. La función `limpiar_clasificacion` es agresiva y alimenta a los modelos. La función `limpiar_sentimiento` es conservadora y alimenta a VADER. Usar una sola de las dos para todo rompe una de las dos tareas, y más adelante se muestra con números cuánto la rompe.

Pasos que se aplicaron

La limpieza para clasificación aplica estos pasos en orden. Corrección de mojibake y de entidades HTML, sustitución de las URL por un token uniforme, eliminación de las menciones, eliminación del símbolo de numeral dejando la palabra del hashtag, paso a minúsculas, eliminación de signos de puntuación, tokenización, eliminación de palabras vacías y lematización.

La Tabla 2 muestra el efecto de cada paso sobre tweets reales del corpus.

Paso	Antes	Después
Entidad HTML	Rene Ablaze & Jacinta	Rene Ablaze & Jacinta
URL a token uniforme	Wholesale Markets ablaze http://t.co/IHYXEO	Wholesale Markets ablaze urlweb
Mención	@bbcmdt Wholesale Markets ablaze	Wholesale Markets ablaze

Hashtag sin el símbolo	Reason of this #earthquake	Reason of this earthquake
Emoticon fuera	Ablaze for you Lord :D	Ablaze for you Lord

Tabla 2. Efecto de cada paso de limpieza sobre ejemplos reales del corpus.

Dos decisiones se apartan de la receta habitual y las dos vienen de lo que midió el análisis exploratorio. Las URL no se borran, se reemplazan por el token urlweb, porque su sola presencia distingue clases. Los números tampoco se borran, por la misma razón. La sección del análisis exploratorio trae las cifras que sostienen ambas decisiones.

Sobre el 911 que menciona el enunciado, se revisó de forma explícita. Aparece en 4 tweets de los 7,485, o sea que no cambia nada en ninguna dirección.

También se revisó si había emojis Unicode que quitar. No hay ninguno en el corpus. Lo que sí hay son 52 tweets con emoticones de texto, un 0.69% del total, y esos se tratan de forma distinta en cada limpieza. La de clasificación los borra y la de sentimiento los traduce a marcadores que VADER entiende.

Stemming y lematización

Se probaron las dos formas de reducir las palabras a su raíz. El stemming corta el final de la palabra siguiendo reglas, y la lematización busca la forma de diccionario.

La Tabla 3 muestra la diferencia entre las dos sobre palabras típicas de este corpus.

Palabra original	Stemming	Lematización
earthquake	earthquak	earthquake
bodies	bodi	body
evacuated	evacu	evacuated
disasters	disast	disaster
flooding	flood	flooding

Tabla 3. Stemming frente a lematización sobre palabras del corpus.

El stemming produce formas que ya no son palabras, como earthquak o disast. Eso no le molesta a un modelo, pero sí complica leer los coeficientes después. Las dos versiones se guardaron y las dos entraron a la parrilla de modelos. La diferencia entre ellas resultó ser de 0.0004 de macro-F1, o sea prácticamente nada, y se terminó usando la lematización por lo legible.

Palabras vacías

Se parte de la lista de palabras vacías del inglés de NLTK, con dos cambios. Primero, se sacaron de la lista las negaciones, o sea no, not, never, nothing, neither, nor, without y none. El material de clase muestra que quitar el no invierte el sentido de una frase, y aunque la tarea de clasificación es de tema, se dejó consistente con la de sentimiento.

Segundo, se agregaron palabras vacías propias de esta colección. El criterio se fijó antes de mirar la lista resultante. Una palabra entra si aparece en más del 1% de los tweets y su log-ratio entre clases tiene valor absoluto menor a 0.25, o sea que es frecuente pero no distingue nada. Así entraron 11 palabras como video, still, us, world y rt, que es el marcador de retweet.

Los módulos de Python que se usaron en toda esta parte son NLTK para las palabras vacías, la tokenización y la lematización, el módulo re de la biblioteca estándar para las expresiones regulares, y el módulo html para las entidades. El detalle está en src/preprocesamiento.py.

Análisis exploratorio

Distribución de las clases

El corpus está desbalanceado, aunque no de forma grave. Hay 4,297 tweets que no hablan de un desastre real, un 57.41%, y 3,188 que sí, un 42.59%.

La Figura 1 muestra el reparto.

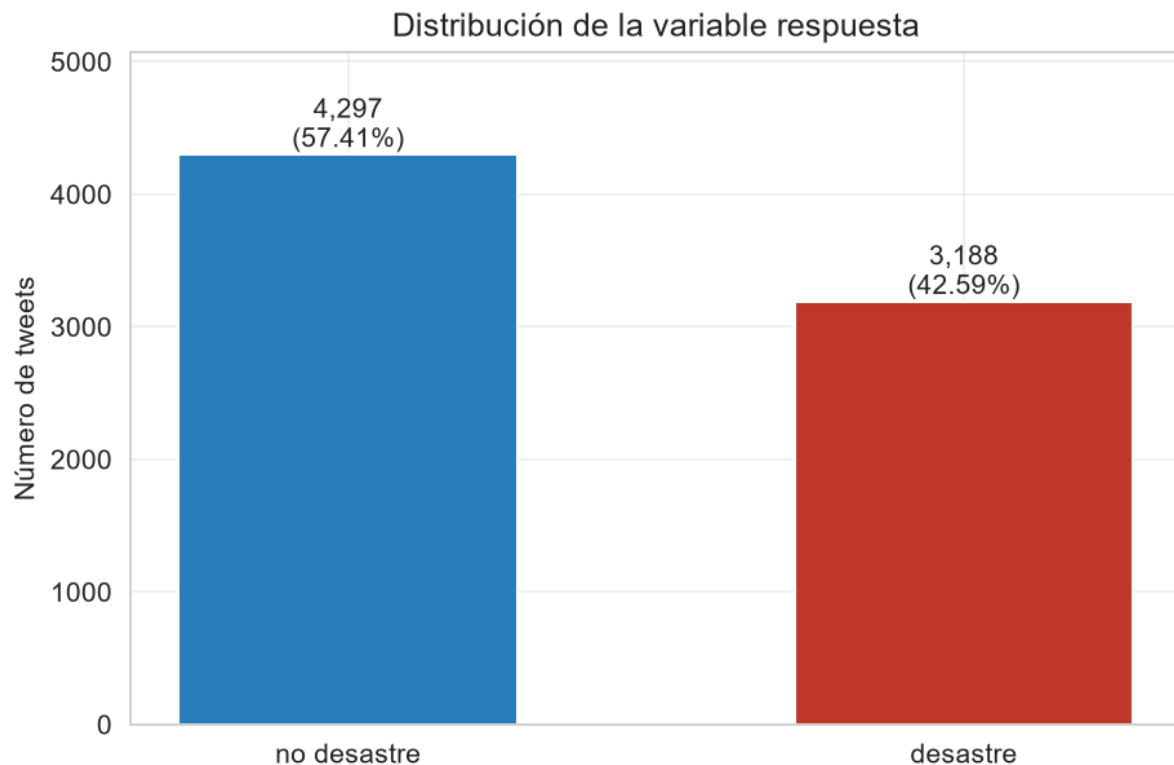


Figura 1. Distribución de las clases en el corpus depurado de 7,485 tweets.

Ese desbalance tiene una consecuencia directa sobre cómo se miden los modelos. Un clasificador que siempre responda la clase mayoritaria acierta el 57.38% de las veces sin haber aprendido nada. Por eso la métrica principal del laboratorio es el macro-F1, que promedia el F1 de las dos clases y castiga a un modelo que ignore la minoritaria.

Patrones del texto

Antes de mirar palabras se midieron patrones de superficie, o sea cosas que se pueden contar sin entender el texto. El resultado fue el hallazgo que más cambió las decisiones de limpieza.

La Tabla 4 recoge cada patrón con su presencia en las dos clases.

Patrón	Global %	No desastre %	Desastre %	Diferencia
Contiene URL	52.64	41.73	67.35	+25.62
Contiene algún dígito	59.63	49.90	72.74	+22.85
Contiene mención (@)	26.64	31.07	20.67	-10.40
Contiene exclamación	9.45	11.99	6.02	-5.96

Contiene hashtag (#)	22.62	20.13	25.97	+5.84
Contiene mayúsculas	19.59	18.27	21.36	+3.09
Contiene entidad HTML	4.77	5.14	4.27	-0.88

Tabla 4. Patrones de superficie por clase. La diferencia está en puntos porcentuales.

La Figura 2 presenta los mismos datos de forma comparada.

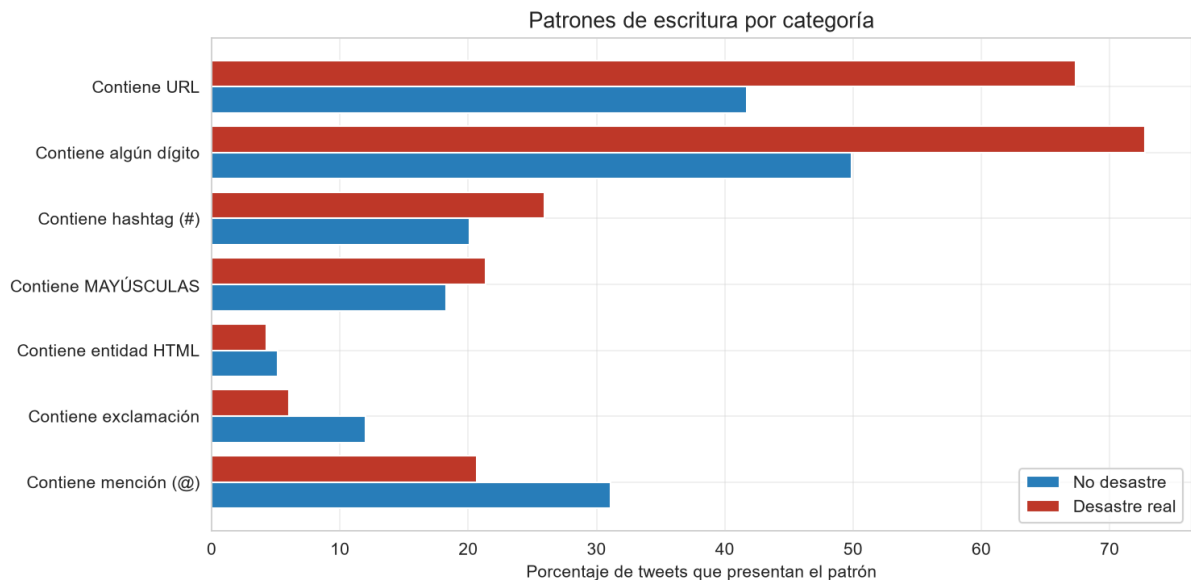


Figura 2. Presencia de cada patrón de superficie en las dos clases.

La presencia de una URL es el patrón más discriminativo de todo el corpus, con 25.62 puntos de diferencia. Tiene sentido, un tweet sobre un desastre real suele enlazar la noticia. Los dígitos vienen justo después con 22.85 puntos, porque los reportes traen cifras de heridos, magnitudes y fechas. Las menciones van al revés, aparecen más en la conversación cotidiana.

Por eso la limpieza no borra las URL ni los números, que es lo que sugiere el enunciado como opción. Borrar la URL tiraría a la basura la señal más fuerte que hay. Reemplazarla por un token uniforme conserva el hecho de que había un enlace sin arrastrar el ruido de la dirección concreta, que nunca se repite.

Longitud de los tweets

Los tweets de desastre son un poco más largos en caracteres, 108.02 de media contra 95.58, y también menos dispersos, con desviación de 29.27 contra 35.94. En número de palabras la diferencia casi desaparece, 15.14 contra 14.67, y la mediana es 15 en las dos clases.

La Figura 3 muestra las dos distribuciones de longitud superpuestas.

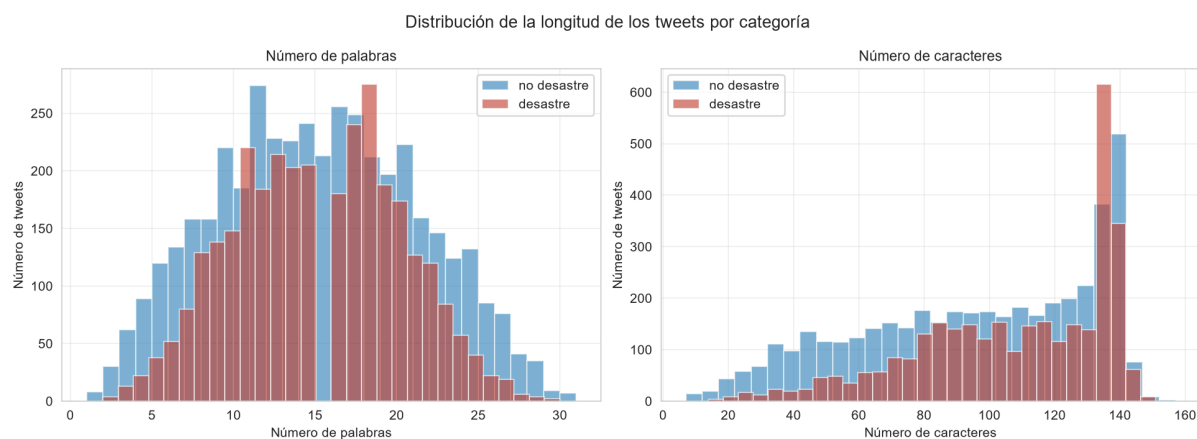


Figura 3. Histogramas de longitud en palabras y en caracteres por clase.

La lectura es que los tweets de desastre usan palabras más largas y un tono más informativo, mientras que los otros se reparten de forma más despareja entre respuestas cortas y textos largos. Por sí sola la longitud no separa las clases, pero apoya la idea de que hay un registro periodístico concentrado en una de ellas.

La palabra clave

La columna keyword resultó ser la variable más informativa del conjunto. Al cruzarla con la clase aparecen extremos casi deterministas. Palabras como wreckage, derailment y debris son desastre en el 100% de los casos, y en el otro extremo hay keywords que nunca lo son.

La Tabla 5 recoge las ocho keywords con mayor tasa de desastre.

Keyword	Tweets	Tasa de desastre
wreckage	37	1.000
derailment	33	1.000
debris	37	1.000
outbreak	39	0.974
oil spill	38	0.974
typhoon	36	0.972
suicide bombing	32	0.969
nuclear disaster	34	0.912

Tabla 5. Las keywords con mayor tasa de desastre. Hay 221 keywords distintas.

La Figura 4 extiende esa mirada a los dos extremos a la vez.

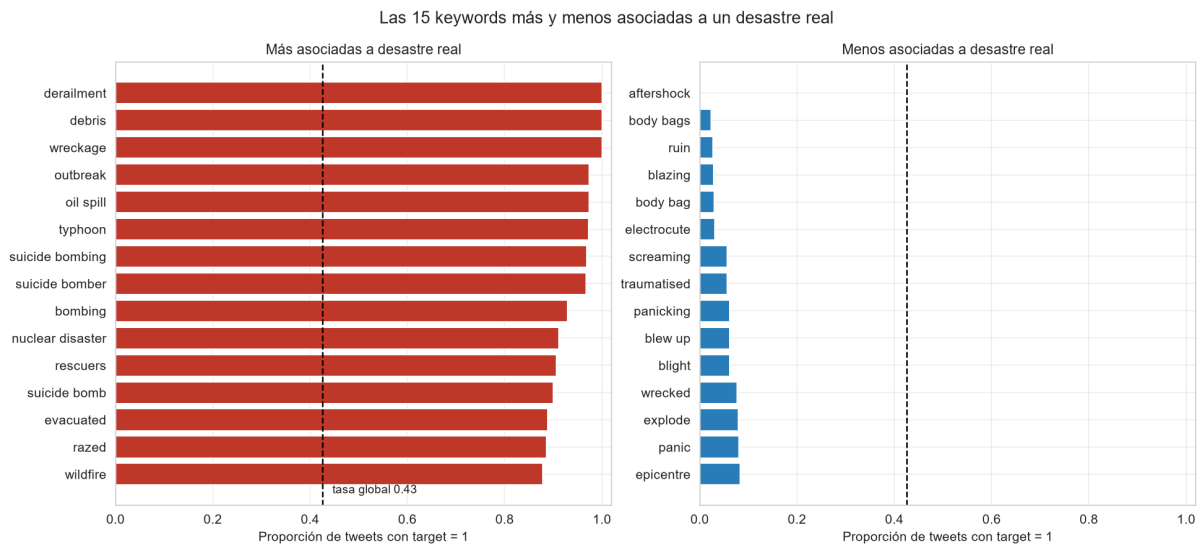


Figura 4. Tasa de desastre de las keywords más extremas en los dos sentidos.

La parte interesante de esa figura está en el medio. La zona donde la tasa ronda 0.5 es donde la palabra clave no decide nada, y ahí es donde se puede anticipar que un clasificador va a fallar. Más adelante se comprueba con el modelo ya entrenado, y la predicción se cumple.

Nubes de palabras

El enunciado pide de forma explícita una nube de palabras que ayude a detectar las que más se repiten. Se generaron tres, una del corpus completo y una por clase, todas sobre el texto ya limpio y lematizado para que no se llenen de artículos.

La Figura 5 corresponde al corpus completo.



Figura 5. Nube de palabras del corpus completo sobre el texto ya limpio.

La Figura 6 toma solo los tweets de desastre real.

[illegible]

La Figura 7 toma los que no describen ningún desastre.

[illegible]

Las dos nubes por clase se ven distintas de un vistazo. La de desastre se llena de fire, news, killed, disaster, california y hiroshima. La otra se llena de like, get, body, love y day. La palabra fire aparece en las dos, y ese solapamiento es justo el problema que después se ve en los errores del modelo.

La Figura 8 compara las palabras más frecuentes de cada clase una al lado de la otra.

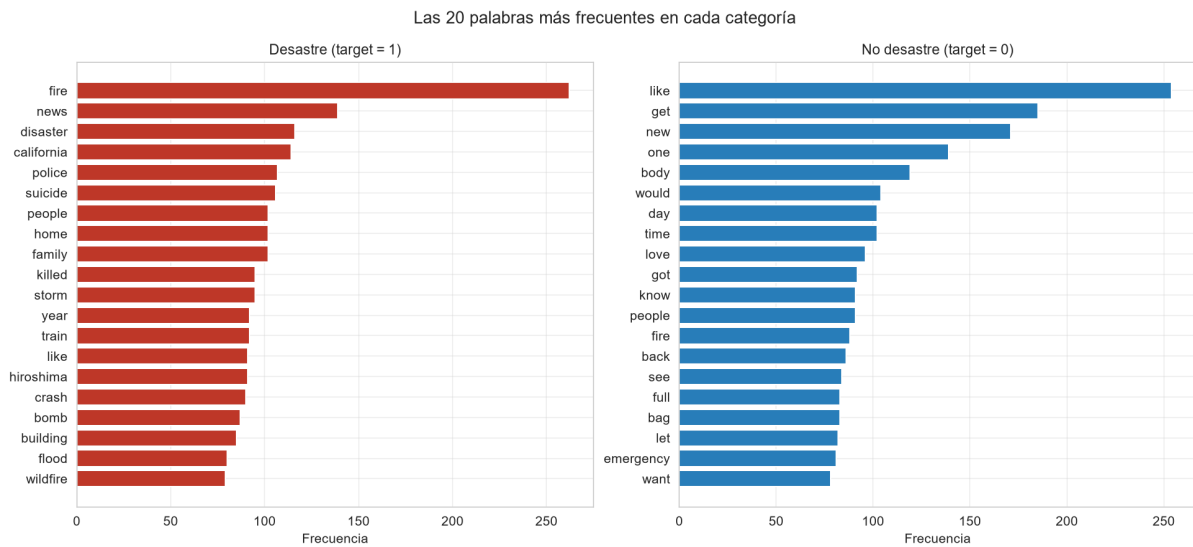


Figura 8. Palabras más frecuentes en cada clase sobre el texto limpio.

El enunciado pide discutir las palabras que tienen presencia en todas las categorías, y ahí está la parte más útil del análisis. Se toman palabras que aparecen bastante en las dos clases y se ordenan por su log-ratio, que es el logaritmo del cociente entre la probabilidad de la palabra en desastre y en no desastre. Un valor cerca de cero quiere decir que la palabra no ayuda a decidir.

El resultado está en la Tabla 6.

Palabra	Frec. desastre	Frec. no desastre	Log-ratio
fire	262	88	+1.263
news	139	58	+1.046
year	92	65	+0.520
people	102	91	+0.286
emergency	77	81	+0.122
time	59	102	-0.375
one	70	139	-0.514
get	70	185	-0.800
like	91	254	-0.854

Tabla 6. Palabras presentes en las dos clases, ordenadas por su log-ratio.

El caso de emergency es el más revelador. Aparece 77 veces en desastre y 81 en no desastre, con un log-ratio de +0.122 que es prácticamente cero. Una palabra que suena a catástrofe resulta inútil para clasificar, porque la gente la usa igual para una emergencia de verdad que para una reunión urgente del trabajo. La palabra fire va en el otro sentido, aparece en las dos clases pero con una proporción muy despareja, y por eso sí sirve.

N-gramas, frecuencias y probabilidades

Cómo se generaron

Los n-gramas se generaron sobre el texto ya limpio y lematizado, recorriendo los tokens de cada tweet en ventanas de tamaño n. Para los unigramas se cuenta cada token, para los bigramas cada par consecutivo y para los trigramas cada terna. El conteo se hizo por separado en cada clase, porque el interés es comparar las dos.

La probabilidad de un n-grama dentro de una clase se calcula como su frecuencia dividida entre el total de n-gramas de esa clase. Es una estimación de máxima verosimilitud, que es la forma directa de estimar $P(\text{n-grama} \mid \text{clase})$. Los n-gramas nunca cruzan de un tweet a otro, cada tweet se procesa como una secuencia independiente.

Unigramas

La Tabla 7 recoge los unigramas más frecuentes de la clase de no desastre con su probabilidad estimada.

Unigrama	Clase	Frecuencia	Probabilidad
urlweb	no desastre	2,193	0.056576
n't	no desastre	326	0.008410
like	no desastre	254	0.006553
not	no desastre	207	0.005340
get	no desastre	185	0.004773
new	no desastre	171	0.004412

Tabla 7. Unigramas más frecuentes en la clase de no desastre, con su probabilidad.

La Figura 9 compara las dos clases.

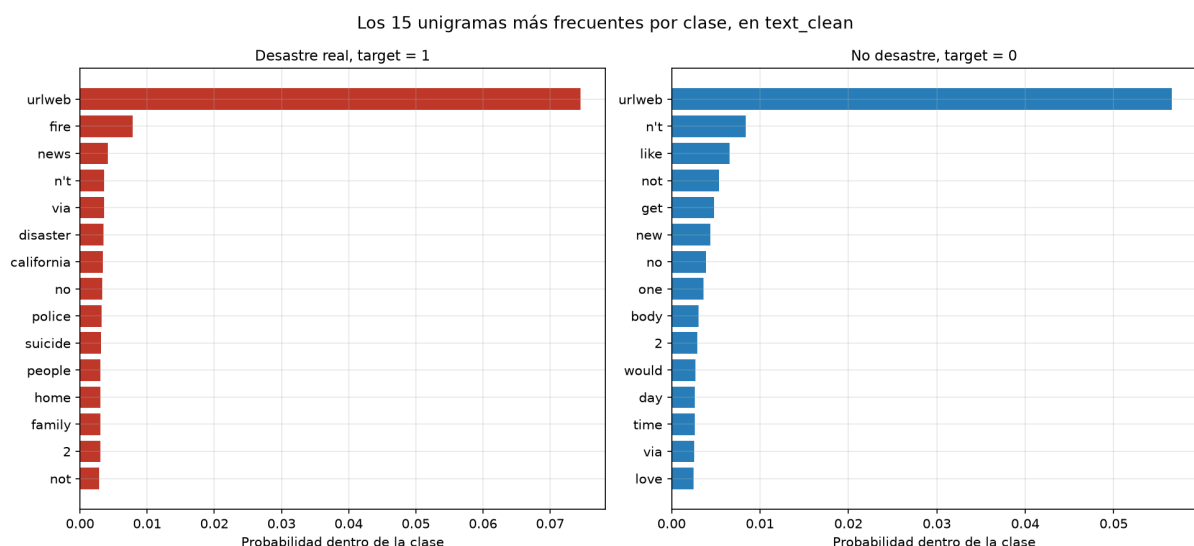


Figura 9. Unigramas más frecuentes en cada clase sobre el texto limpio.

El token urlweb domina las dos listas, lo que confirma desde otro ángulo lo que ya mostró el análisis de patrones. Después de él aparecen las contracciones y las negaciones que se conservaron a propósito. En la clase de desastre las primeras posiciones las ocupan fire, news y killed.

Bigramas y trigramas

El enunciado pregunta si vale la pena explorar bigramas o trigramas para analizar contexto. La respuesta corta, adelantándose a la sección de modelos, es que sirven para entender el corpus y casi no sirven para clasificar. Conviene ver primero qué salió.

La Tabla 8 recoge los bigramas más frecuentes de la clase de no desastre.

Bigrama	Clase	Frecuencia	Probabilidad
urlweb urlweb	no desastre	284	0.008240

ca n't	no desastre	69	0.002002
body bag	no desastre	49	0.001422
cross body	no desastre	39	0.001132
look like	no desastre	32	0.000928
burning building	no desastre	19	0.000551

Tabla 8. Bigramas más frecuentes en la clase de no desastre.

La Figura 10 compara los bigramas de las dos clases.

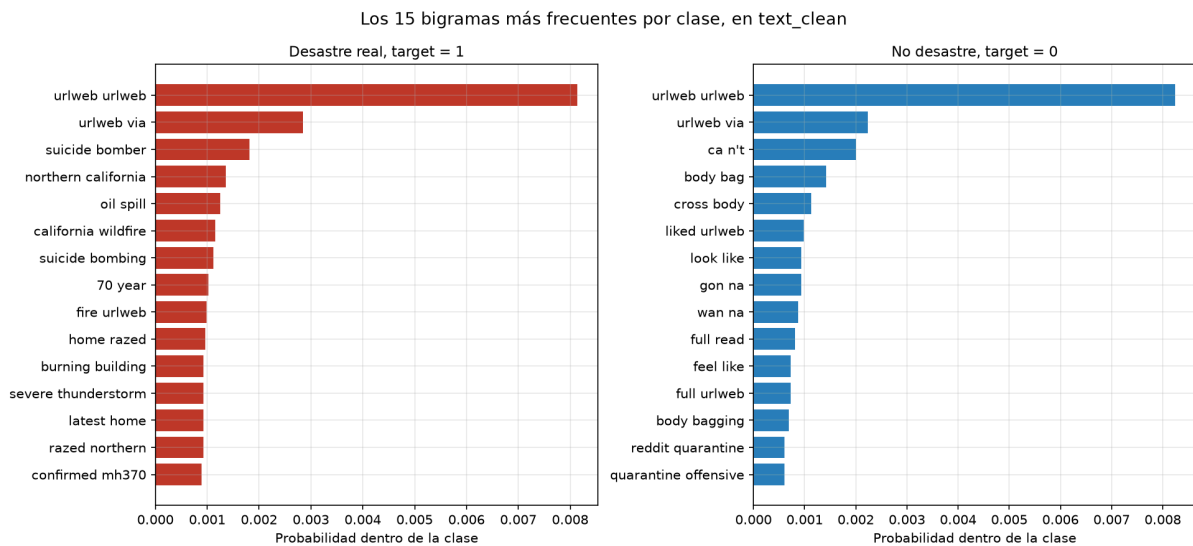


Figura 10. Bigramas más frecuentes en cada clase.

El bigrama body bag es un buen ejemplo de por qué el contexto importa. La palabra body sola aparece 119 veces en la clase de no desastre y suena macabra, pero el bigrama muestra que la mayoría de las veces se trata de bolsos cruzados que se venden por internet, no de bolsas para cadáveres. Un unigrama nunca hubiera podido mostrar eso.

La Tabla 9 hace lo mismo con los trigramas.

Trigrama	Clase	Frecuencia	Probabilidad
full urlweb urlweb	no desastre	25	0.000827
cross body bag	no desastre	24	0.000794
reddit quarantine offensive	no desastre	21	0.000695
china stock market	no desastre	16	0.000529
stock market crash	no desastre	16	0.000529
ignition knock detonation	no desastre	15	0.000496

Tabla 9. Trigramas más frecuentes en la clase de no desastre.

La Figura 11 los compara entre clases.

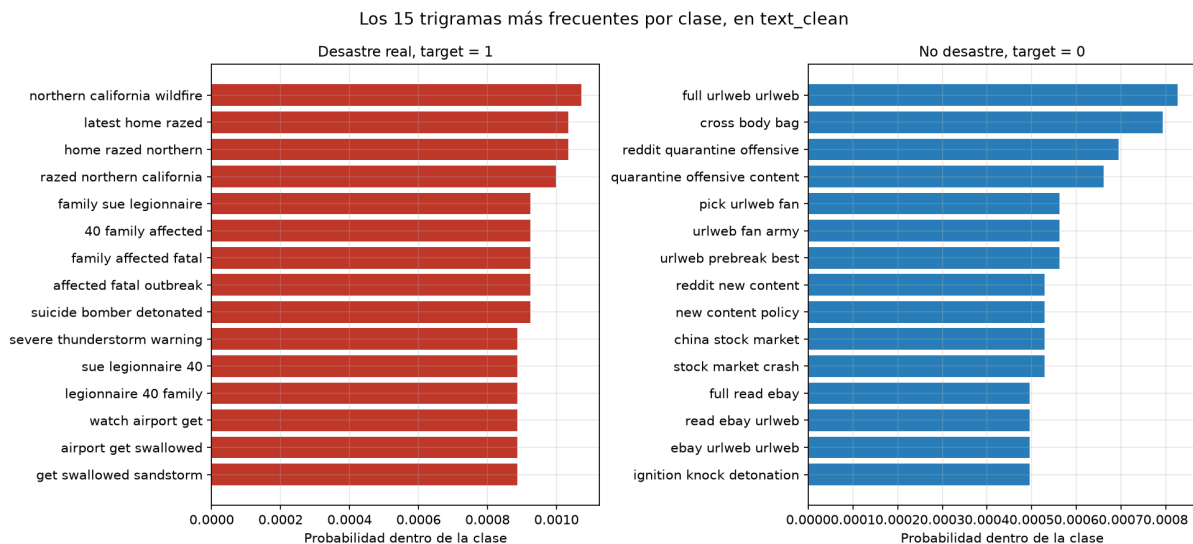


Figura 11. Trigramas más frecuentes en cada clase.

Los trigramas ya muestran el problema. Casi todos son frases hechas de una campaña, un producto o una noticia concreta, como ignition knock detonation sensor que viene de anuncios de repuestos de carro. Frecuencias de 15 o 16 sobre miles de tweets no alcanzan para generalizar a un tweet nuevo.

Probabilidad condicional

Además de la probabilidad de cada n-grama se calculó la probabilidad condicional del segundo término dado el primero, o sea $P(w_2 \text{ dado } w_1)$, que es el conteo del par dividido entre el conteo de w_1 . Sirve para ver qué tan pegadas van dos palabras.

La Tabla 10 recoge los bigramas donde esa probabilidad es más alta.

Bigrama	Conteo del par	Conteo de w_1	$P(w_2 w_1)$
wan na	35	35	1.0000
prebreak best	30	30	1.0000
liked urlweb	41	44	0.9318
gon na	43	47	0.9149
razed northern	28	33	0.8485
ca n't	88	111	0.7928

Tabla 10. Bigramas con mayor probabilidad condicional en el corpus.

Los primeros lugares los ocupan artefactos de la tokenización, como wan na y gon na, que salen de partir wanna y gonna. Eso sube a 1.0 porque wan solo existe dentro de wanna. El caso de razed northern es distinto y sí es real, viene de una noticia repetida sobre un pueblo arrasado en el norte, y muestra cómo un evento concreto deja huella en el corpus.

Qué palabras sirven para clasificar

El enunciado pregunta qué palabras se cree que servirán para hacer un mejor modelo. Para responderlo con números y no con intuición se calculó el log-ratio de cada palabra entre las dos clases, con suavizado de Laplace para que las palabras que aparecen en una sola clase no den infinito.

La Tabla 11 recoge las que más empujan hacia la clase de desastre.

Palabra	Frec. desastre	Frec. no desastre	Log-ratio
---------	----------------	-------------------	-----------

mh370	70	0	+4.370
northern	61	0	+4.235
legionnaire	60	0	+4.219
debris	49	0	+4.020
hiroshima	91	1	+3.936
derailment	35	0	+3.691
typhoon	56	1	+3.458
wildfire	79	2	+3.391

Tabla 11. Palabras que más empujan hacia la clase de desastre según su log-ratio.

La Tabla 12 hace lo mismo hacia el otro lado.

Palabra	Frec. no desastre	Frec. desastre	Log-ratio
bag	83	2	-3.224
ruin	39	1	-2.888
lmao	16	0	-2.726
ebay	29	1	-2.600
electrocute	28	1	-2.566
handbag	24	1	-2.418
cake	22	1	-2.335
blew	32	2	-2.290

Tabla 12. Palabras que más empujan hacia la clase de no desastre.

La Figura 12 pone las dos listas frente a frente.

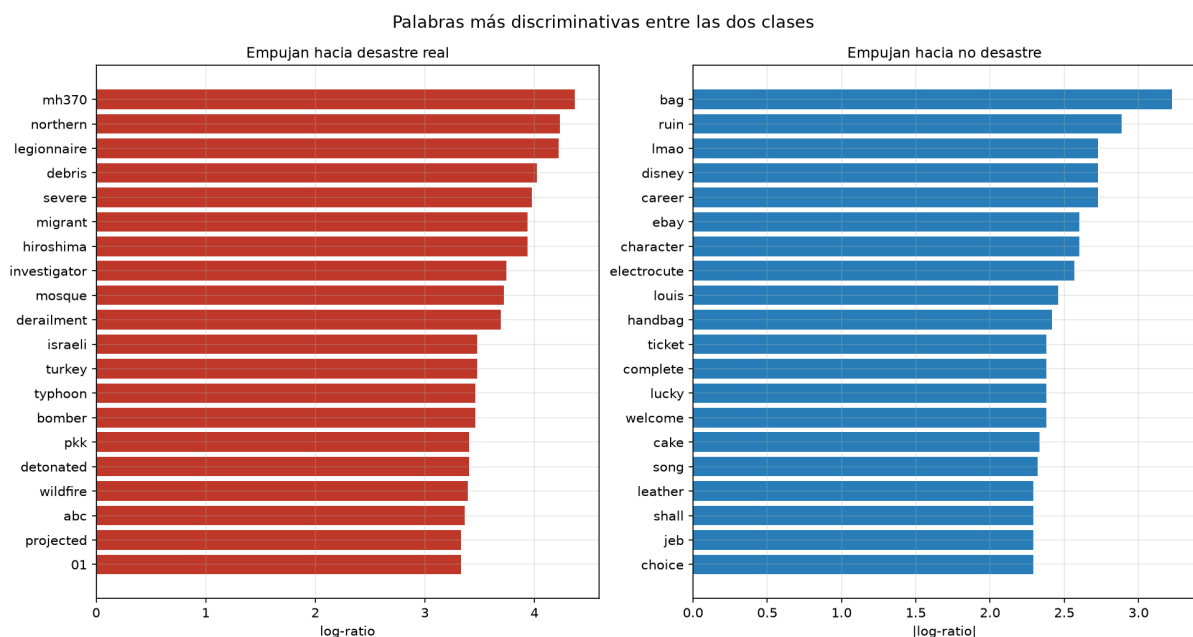


Figura 12. Palabras con mayor log-ratio hacia cada una de las dos clases.

Las palabras que empujan hacia desastre son casi todas nombres propios de eventos concretos. mh370 es el vuelo desaparecido de Malaysia Airlines, hiroshima aparece por el aniversario del bombardeo, legionnaire viene de un brote de legionela. Sirven muchísimo dentro de este corpus y no van a servir en tweets de otro periodo, y eso es una limitación real del modelo que conviene tener presente.

Las que empujan hacia el otro lado son las más interesantes para entender el problema. Palabras como ruin, electrocute y blew son vocabulario de catástrofe y aparecen del lado de no desastre. La razón es que la gente las usa en sentido figurado, del tipo esta película arruinó mi día. Ese uso figurado es el que después explica la mitad de los errores del clasificador.

También se buscaron las palabras más neutras, o sea las que tienen frecuencia parecida en las dos clases. Ahí aparecen fatality con log-ratio de 0.0023, weapon con -0.0048 y emergency con 0.0577. Son palabras que suenan decisivas y no aportan nada.

Modelos de clasificación

Cómo se abordó el contexto

El enunciado pregunta de forma directa cómo se planea abordar el contexto. La respuesta de este trabajo tiene tres partes y las tres se pusieron a prueba con experimentos, no se dieron por buenas.

- Contexto dentro del texto, agregando bigramas y trigramas al vectorizador para que el modelo vea pares y ternas de palabras además de palabras sueltas.
- Contexto por debajo de la palabra, con n-gramas de caracteres, que capturan raíces compartidas y aguantan mejor las erratas típicas de Twitter.
- Contexto fuera del texto, metiendo la columna keyword como variable adicional del modelo.

Cada una de las tres es una variante que se entrenó y se midió por separado, y los resultados están más abajo.

Diseño del experimento

Se armó una parrilla de catorce combinaciones cruzando tres algoritmos, dos representaciones del texto y dos formas de reducir las palabras, más Random Forest aparte. Los algoritmos son regresión logística, SVM lineal y ComplementNB, que es la variante de Naive Bayes pensada para clases desbalanceadas. Las representaciones son bolsa de palabras con CountVectorizer y TF-IDF con TfidfVectorizer, las dos con unigramas y bigramas y con min_df igual a 2 para descartar términos que aparecen una sola vez.

Dos cuidados sostienen todo lo que sigue. El primero es que el vectorizador va siempre dentro de un Pipeline de scikit-learn, así que se ajusta solo con el conjunto de entrenamiento y nunca ve el de prueba. Si se ajustara sobre todo el corpus, el vocabulario y los pesos IDF traerían información del conjunto de prueba y las métricas saldrían infladas. El segundo es que los modelos que lo permiten llevan class_weight balanceado, para que la clase minoritaria pese lo mismo que la otra.

La parrilla completa

La Tabla 13 recoge las catorce combinaciones ordenadas por macro-F1 sobre el conjunto de prueba.

Modelo	Exactitud	F1 desastre	Recall desastre	Macro-F1
BOW + ComplementNB (stem)	0.7963	0.7395	0.6787	0.7861
TF-IDF + Reg. logística (lema)	0.7923	0.7482	0.7241	0.7857
TF-IDF + ComplementNB (lema)	0.7943	0.7368	0.6755	0.7839

TF-IDF + Reg. logística (stem)	0.7896	0.7437	0.7163	0.7826
TF-IDF + ComplementNB (stem)	0.7896	0.7310	0.6708	0.7791
BOW + ComplementNB (lema)	0.7889	0.7317	0.6755	0.7789
BOW + Reg. logística (lema)	0.7856	0.7318	0.6865	0.7766
TF-IDF + SVM lineal (lema)	0.7769	0.7289	0.7038	0.7697
BOW + Reg. logística (stem)	0.7782	0.7219	0.6755	0.7687
BOW + Random Forest (lema)	0.7776	0.7199	0.6708	0.7677
TF-IDF + Random Forest (lema)	0.7709	0.7091	0.6552	0.7600
TF-IDF + SVM lineal (stem)	0.7655	0.7153	0.6912	0.7580
BOW + SVM lineal (stem)	0.7609	0.7089	0.6834	0.7530
BOW + SVM lineal (lema)	0.7602	0.7036	0.6677	0.7511

Tabla 13. Las catorce combinaciones de la parrilla, ordenadas por macro-F1 sobre el conjunto de prueba.

La Figura 13 muestra lo mismo de un vistazo.

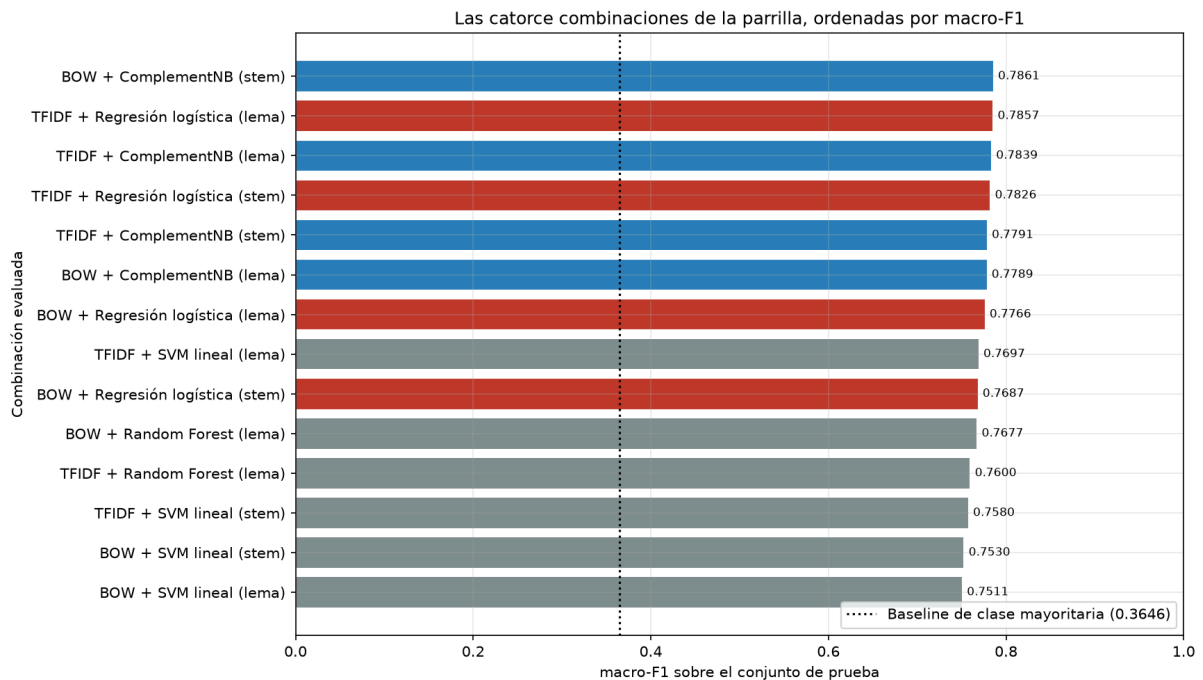


Figura 13. Macro-F1 de las catorce combinaciones de la parrilla.

Ahí está el resultado que ordena toda esta sección. La mejor combinación llega a 0.7861 y la peor a 0.7511, o sea que las catorce caben dentro de 0.0350 de macro-F1. Todas superan por mucho al

clasificador trivial, que se queda en 0.3646, pero entre ellas la diferencia es chica. Elegir un algoritmo u otro no es donde se gana en este problema.

Random Forest quedó en la parte baja, lo que confirma la advertencia del material de clase sobre los árboles con matrices dispersas. Su mejor combinación llega a 0.7677 y además es la más cara de entrenar de todas.

Las variantes de contexto

La Tabla 14 recoge las cinco variantes con el tamaño de vocabulario que genera cada una.

Variante	Vocabulario	Recall desastre	Macro-F1
Solo unigramas (1,1)	4,966	0.7226	0.7856
Unigramas y bigramas (1,2)	9,308	0.7241	0.7857
Uni, bi y trigramas (1,3)	12,142	0.7304	0.7893
N-gramas de caracteres (3,5)	25,514	0.6975	0.7743
TF-IDF (1,2) con keyword	9,308	0.7194	0.7669

Tabla 14. Las cinco variantes de contexto, todas con regresión logística.

La Figura 14 las compara.

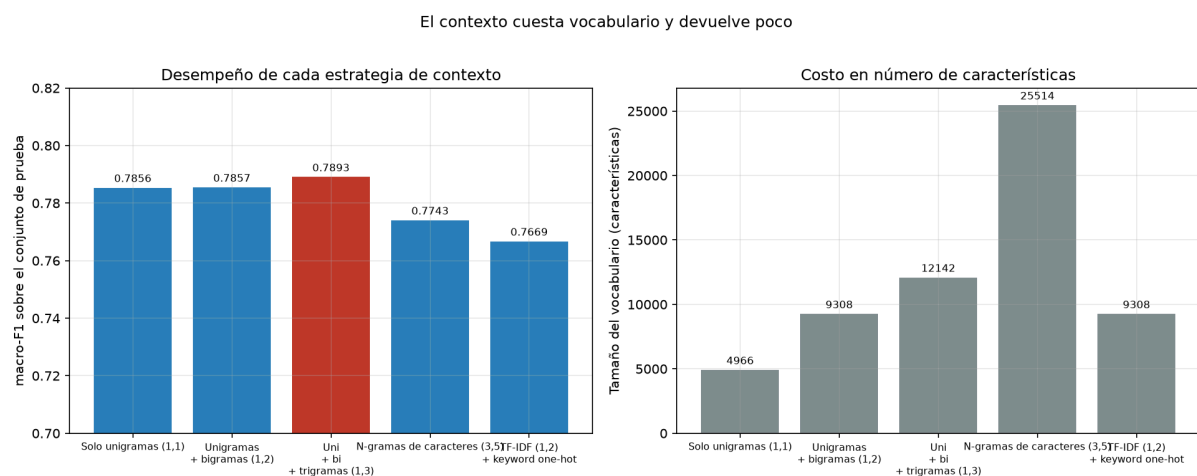


Figura 14. Comparación de las cinco variantes de contexto.

Las tres respuestas quedan claras. Agregar bigramas mejora 0.0001, o sea nada, aunque casi duplica el vocabulario de 4,966 a 9,308 términos. Agregar trigramas mejora 0.0036, que sigue siendo poco para 12,142 términos. Los n-gramas de caracteres empeoran 0.0113 sobre el conjunto de prueba, y más adelante se ve que en validación cruzada hacen justo lo contrario.

La variante con keyword es la que más sorprende, porque empeora 0.0187 siendo que keyword era la variable más informativa del análisis exploratorio. La explicación está en el propio corpus. La keyword ya aparece escrita dentro del texto del tweet en el 94.10% de los casos, entonces el one-hot solo agrega 222 columnas dispersas con información que el vectorizador ya tenía, y el modelo tiene más parámetros que ajustar con los mismos datos.

Validación cruzada y criterio de selección

Comparar sobre el conjunto de prueba y después elegir el mejor tiene un problema. El número que se reporta ya incluye la ventaja de haber escogido el modelo que mejor le cae a esa partición concreta. Por eso los seis mejores candidatos se volvieron a medir con validación cruzada estratificada de cinco pliegues, usando solo el conjunto de entrenamiento.

El criterio de desempate se escribió antes de ver los resultados, para no acomodarlo después. Primero macro-F1 en validación cruzada. Si dos modelos quedan dentro de una desviación estándar, gana el de mayor exhaustividad sobre la clase de desastre, porque dejar pasar un desastre real cuesta más que una falsa alarma. Si siguen empatados, gana el más simple de interpretar.

La Tabla 15 trae el resultado de esa validación cruzada.

Modelo	Macro-F1 CV	Desviación	Mínimo	Máximo	Recall desastre
Caracteres (3,5) + LogReg	0.8011	0.0101	0.7834	0.8130	0.7604
TF-IDF (1,2) + LogReg	0.7974	0.0173	0.7662	0.8171	0.7655
TF-IDF (1,3) + LogReg	0.7956	0.0159	0.7680	0.8139	0.7655
TF-IDF (1,2) + ComplementNB	0.7955	0.0064	0.7847	0.8022	0.7137
BOW (1,2) + ComplementNB	0.7927	0.0098	0.7820	0.8098	0.7133
TF-IDF (1,2) + LogReg (stem)	0.7924	0.0106	0.7778	0.8056	0.7600

Tabla 15. Validación cruzada de cinco pliegues sobre el conjunto de entrenamiento.

La Figura 15 muestra la dispersión entre pliegues de cada candidato.

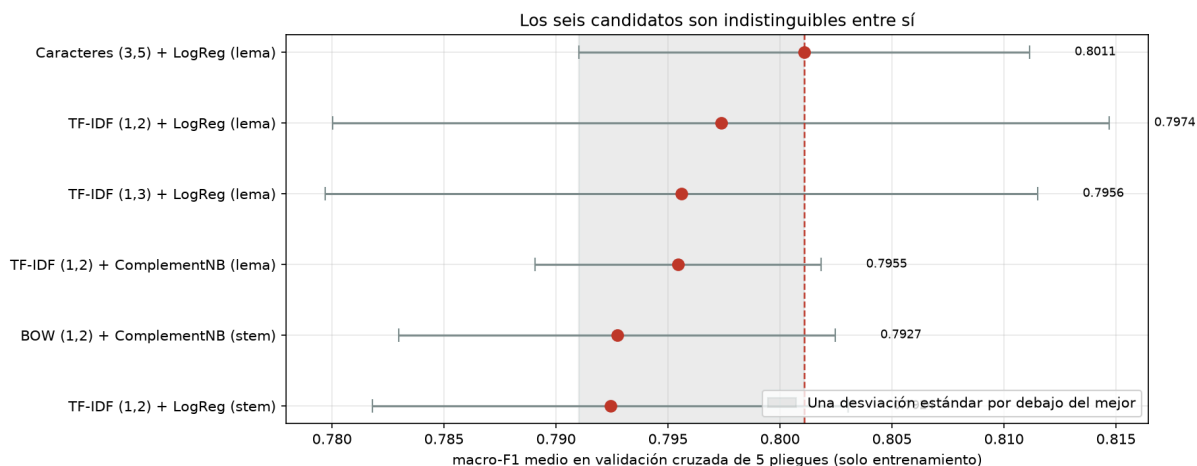


Figura 15. Macro-F1 en validación cruzada de los seis candidatos, con su dispersión.

Los seis candidatos van de 0.7924 a 0.8011, o sea 0.0087 de diferencia, cuando las desviaciones por pliegue van de 0.0064 a 0.0173. Todos caen dentro de una desviación del mejor, así que el primer criterio no alcanza y hay que aplicar el desempate.

Por exhaustividad de la clase de desastre gana TF-IDF (1,2) con regresión logística, con 0.7655 contra 0.7604 del de caracteres. Entonces el modelo seleccionado es TF-IDF con unigramas y bigramas, min_df igual a 2 y regresión logística con pesos balanceados, sobre texto limpio y lematizado. Es

además el mismo modelo que se había elegido de forma preliminar en el avance, y el más simple de leer de los seis.

El caso de los n-gramas de caracteres merece una nota. Son los mejores en validación cruzada con 0.8011 y los peores de las variantes de contexto sobre el conjunto de prueba con 0.7743. Esa contradicción es la mejor prueba que hay de que estas diferencias caen dentro del ruido de muestreo.

Desempeño del modelo seleccionado

La Tabla 16 compara el modelo elegido contra el punto de referencia mínimo, que es el clasificador trivial.

	Macro-F1	Exactitud	Recall desastre	Precisión desastre
Clasificador trivial	0.3646	57.38%	0.0000	0.0000
Modelo seleccionado	0.7857	79.23%	0.7241	0.7739

Tabla 16. Modelo seleccionado frente al clasificador que siempre responde la clase mayoritaria.

La Tabla 17 desglosa los aciertos y los errores por clase.

	Predice no desastre	Predice desastre	Total
Real no desastre	724	135	859
Real desastre	176	462	638
Total	900	597	1,497

Tabla 17. Matriz de confusión del modelo seleccionado sobre los 1,497 tweets de prueba.

La Figura 16 presenta la misma matriz de forma visual.

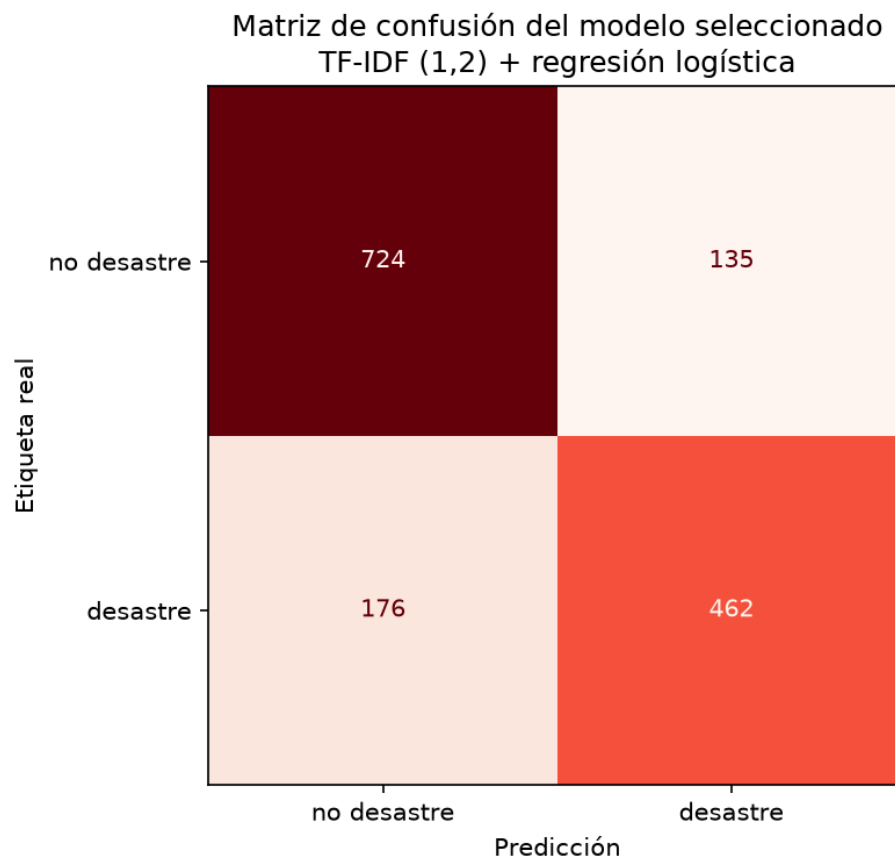


Figura 16. Matriz de confusión del modelo seleccionado.

El modelo comete 311 errores sobre 1,497 tweets. De esos, 176 son falsos negativos, o sea desastres reales que se le escapan, y 135 son falsos positivos. El desbalance entre los dos tipos de error tiene sentido con la clase minoritaria, aunque el peso balanceado ya lo compensa bastante.

Qué aprendió el modelo

La regresión logística tiene la ventaja de que se puede leer por dentro. Cada término del vocabulario tiene un coeficiente, y el signo dice hacia qué clase empuja.

La Tabla 18 recoge los ocho términos más fuertes de cada lado.

Empuja hacia desastre	Coef.	Empuja hacia no desastre	Coef.
fire	+3.54	love	-1.94
hiroshima	+3.17	new	-1.87
storm	+2.63	ruin	-1.72
california	+2.61	bag	-1.70
wildfire	+2.53	wrecked	-1.66
flood	+2.38	traumatised	-1.60
urlweb	+2.26	blew	-1.55
disaster	+2.09	avalanche	-1.49

Tabla 18. Términos con mayor coeficiente hacia cada clase en el modelo seleccionado.

La Figura 17 extiende la lista a veinte términos por clase.

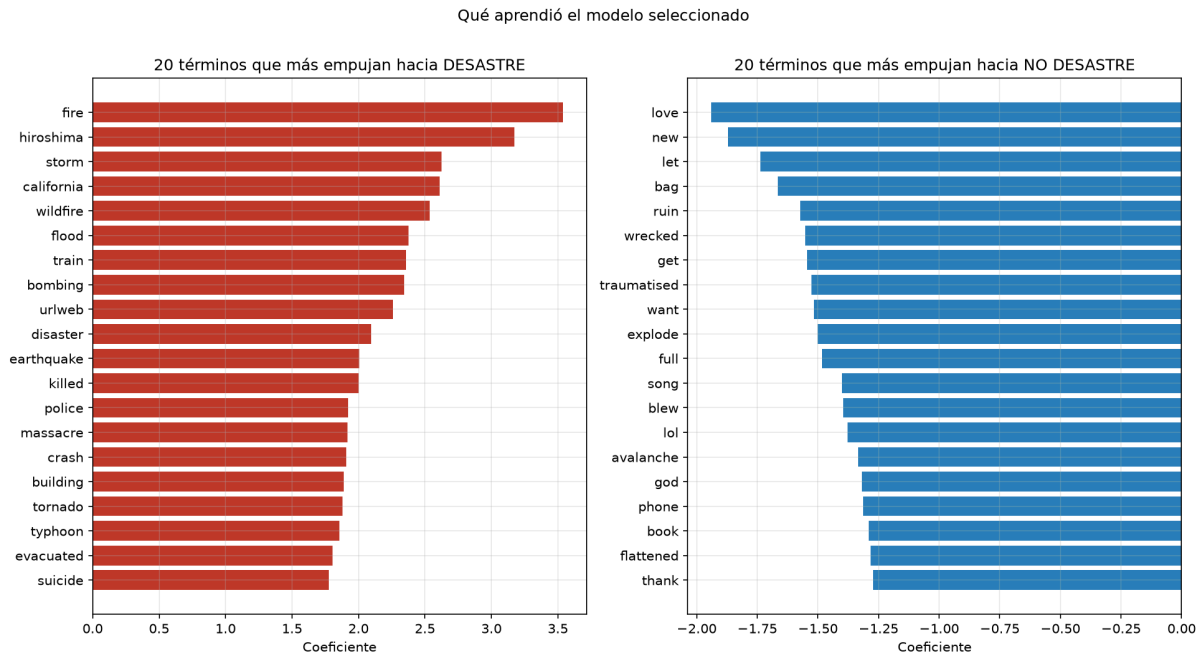


Figura 17. Los veinte términos de mayor coeficiente hacia cada clase.

Dos cosas saltan a la vista. La primera es que urlweb está en el noveno lugar con +2.26, o sea que la decisión de no borrar las URL se paga sola. La segunda es más interesante. Del lado de no desastre aparecen palabras de catástrofe como ruin, wrecked, traumatised, blew y avalanche. El modelo

aprendió por su cuenta que ese vocabulario concreto es marcador de uso figurado, porque es el que la gente usa como metáfora.

Un dato que refuerza lo de los bigramas. El 46.6% del vocabulario del modelo son bigramas, o sea 4,342 de 9,308 términos, y ninguno aparece entre los veinte coeficientes más fuertes de ninguna de las dos clases. El modelo decide con palabras sueltas.

Como comprobación adicional se comparó el coeficiente aprendido de cada término contra el log-ratio que se había calculado a mano en la sección de n-gramas, sobre los 2,549 términos que comparten. La correlación de Spearman entre las dos medidas es 0.8590. Dos métodos que no se hablan entre sí llegan casi a la misma ordenación de palabras.

Análisis de errores

Para entender de qué se equivoca el modelo se tomó una muestra aleatoria de diez falsos positivos y diez falsos negativos y se clasificó cada caso a mano en categorías.

La Tabla 19 recoge el resultado de esa tipificación.

Tipo de error	Categoría	Casos
Falso negativo	Etiqueta discutible	8
Falso negativo	Ambigüedad genuina	1
Falso negativo	Texto insuficiente	1
Falso positivo	Uso figurado	5
Falso positivo	Contexto inofensivo	2
Falso positivo	Texto insuficiente	2
Falso positivo	Vocabulario de noticias	1

Tabla 19. Tipificación manual de una muestra de veinte errores del modelo.

La Figura 18 muestra el reparto.

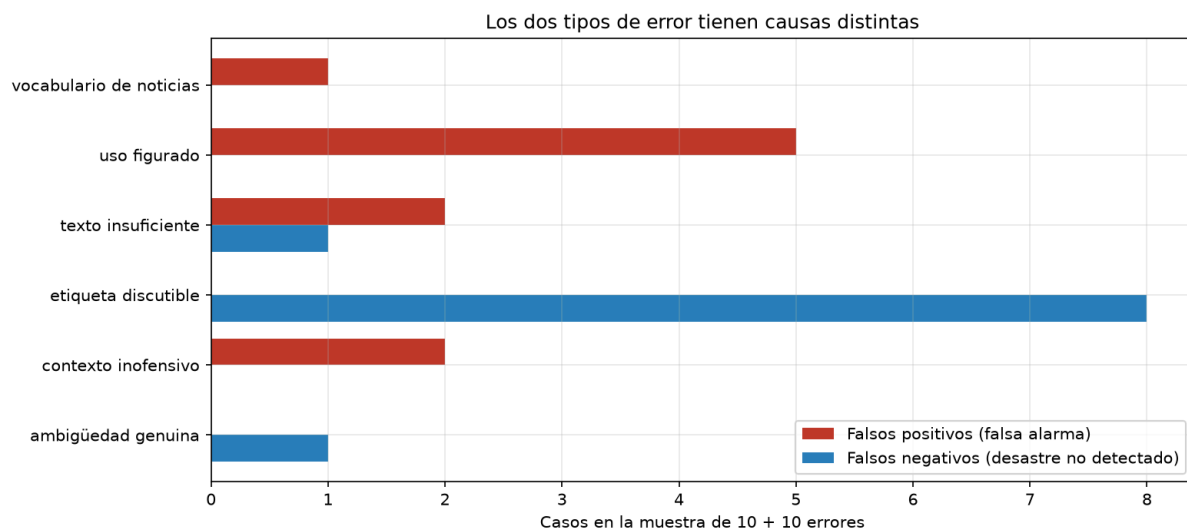


Figura 18. Reparto de los errores tipificados a mano por tipo y categoría.

La asimetría entre los dos tipos de error es el hallazgo. Los falsos positivos son en su mayoría uso figurado, o sea tweets como slicker than an oil spill o my asshole is on fire, donde el vocabulario de catástrofe está usado como metáfora. Ahí el modelo se equivoca de verdad.

Los falsos negativos son otra cosa. Ocho de diez son casos donde la etiqueta del conjunto es la que cuesta defender. Aparecen cosas como Did you get wrecked again?, o I waited 2.5 hours to get a cab my feet are bleeding, o una queja sobre un videojuego que menciona una catástrofe. Todos vienen marcados como desastre real, y el modelo, al responder que no, es difícil de contradecir.

La Tabla 20 trae ejemplos concretos de los dos tipos de error.

Tweet (abreviado)	Etiqueta	Predicción	Confianza
slicker than an oil spill	no desastre	desastre	0.855
I'm a cyclone passion overblown	no desastre	desastre	0.820
My asshole is on fire	no desastre	desastre	0.815
Did you get wrecked again?	desastre	no desastre	0.071
all that panicking made me tired	desastre	no desastre	0.103
I waited 2.5 hours to get a cab my feet are bleeding	desastre	no desastre	0.137

Tabla 20. Ejemplos de errores del modelo. La confianza es la probabilidad estimada de la clase de desastre.

Una parte del error medido es entonces ruido de la anotación y no una falla del modelo. Eso no lo vuelve mejor de lo que dice su métrica, porque no hay forma de saber qué parte del error es de cada tipo sin reanotar el conjunto. Lo que sí explica es por qué catorce combinaciones caben en 0.035 de macro-F1.

Queda una comprobación más. El análisis exploratorio había anticipado que los errores se iban a concentrar en las keywords de tasa intermedia, o sea las que no deciden nada por sí solas. Con el modelo ya entrenado se midió, y la tasa de error es 25.03% en las keywords de tasa intermedia contra 10.34% en las de tasa extrema alta. La predicción se cumplió.

La Figura 19 muestra la tasa de error en los tres grupos de keywords.

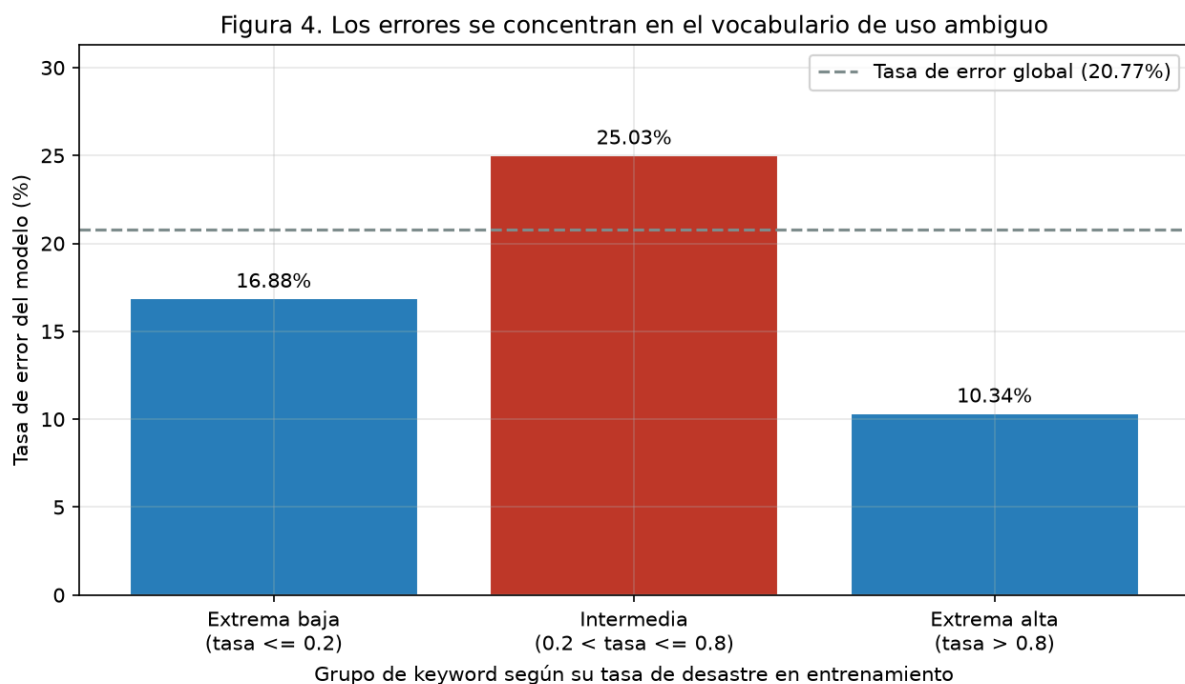


Figura 19. Tasa de error del modelo según qué tan extrema es la keyword del tweet.

Función para clasificar tweets nuevos

Diseño

El enunciado pide una función donde el usuario ingrese un tweet y el sistema lo clasifique. Está en `src/clasificador.py` y se llama `clasificar_tweet`. Recibe el texto sin preprocesar y devuelve un diccionario con el texto original, el texto ya procesado, la predicción, la confianza, el sentimiento y una advertencia cuando corresponde. También acepta una lista de tweets y en ese caso devuelve un DataFrame con una fila por tweet.

Tres decisiones de diseño valen la pena explicarse.

Todo el preprocesamiento ocurre dentro de la función. Quien la llama escribe el tweet tal como lo escribiría en Twitter, con URL, hashtags, mayúsculas y erratas, y no necesita saber que existen palabras vacías, lematización ni TF-IDF. Si el preprocesamiento quedara por fuera, cada persona que usara el modelo tendría que reproducirlo, y cualquier diferencia mínima produciría predicciones distintas sin ningún aviso.

La función vive en un módulo y no en una celda del cuaderno. Un cuaderno no se puede importar desde otro programa, entonces una función que solo existe dentro de él no es utilizable de verdad. El cuaderno lo comprueba lanzando un proceso de Python nuevo, que no ha ejecutado ninguna celda, y pidiéndole que importe el módulo y clasifique un tweet. El modelo se carga desde `results/modelos/mejor_modelo.joblib`, que guarda el Pipeline completo con el vectorizador ya ajustado y no solo el clasificador.

Los casos borde devuelven una predicción y una advertencia en lugar de lanzar una excepción. Obligar a quien llama a envolver cada uso en un try traslada el problema, y devolver una predicción silenciosa sería peor. La advertencia deja el resultado utilizable y honesto al mismo tiempo.

Prueba con doce tweets propios

Se escribieron doce tweets que no vienen del conjunto de datos. Cuatro claramente de desastre, cuatro claramente ajenos y cuatro diseñados a propósito para engañar al modelo, atacando cada uno una debilidad concreta que el análisis de errores ya había dejado documentada.

La Tabla 21 trae los doce tweets con lo que se esperaba de cada uno y lo que respondió la función.

#	Tweet (abreviado)	Esperado	Predicción	Confianza
1	Magnitude 7.1 earthquake hits central Mexico	desastre	desastre	0.799
2	BREAKING wildfire jumped the highway	desastre	desastre	0.855
3	Flash flood warning until 8pm	desastre	desastre	0.824
4	Death toll from the train derailment rises to 14	desastre	desastre	0.854
5	Just finished my thesis defense, tacos to celebrate	no desastre	no desastre	0.765
6	The new season of that baking show is out	no desastre	no desastre	0.726

7	Reminder, the library closes at 6pm on Fridays	no desastre	desastre	0.522
8	Anyone got recommendations for a running playlist?	no desastre	no desastre	0.796
9	This exam destroyed me, my GPA is a total disaster	no desastre	no desastre	0.632
10	The smoke alarm again, the whole apartment is on fire lol	no desastre	desastre	0.543
11	Oh great, another emergency meeting about the report	no desastre	desastre	0.578
12	No injuries reported, no fire and no evacuation	no desastre	desastre	0.712

Tabla 21. Los doce tweets escritos por el equipo y el resultado de la función.

La Figura 20 muestra el reparto con la confianza de cada predicción.

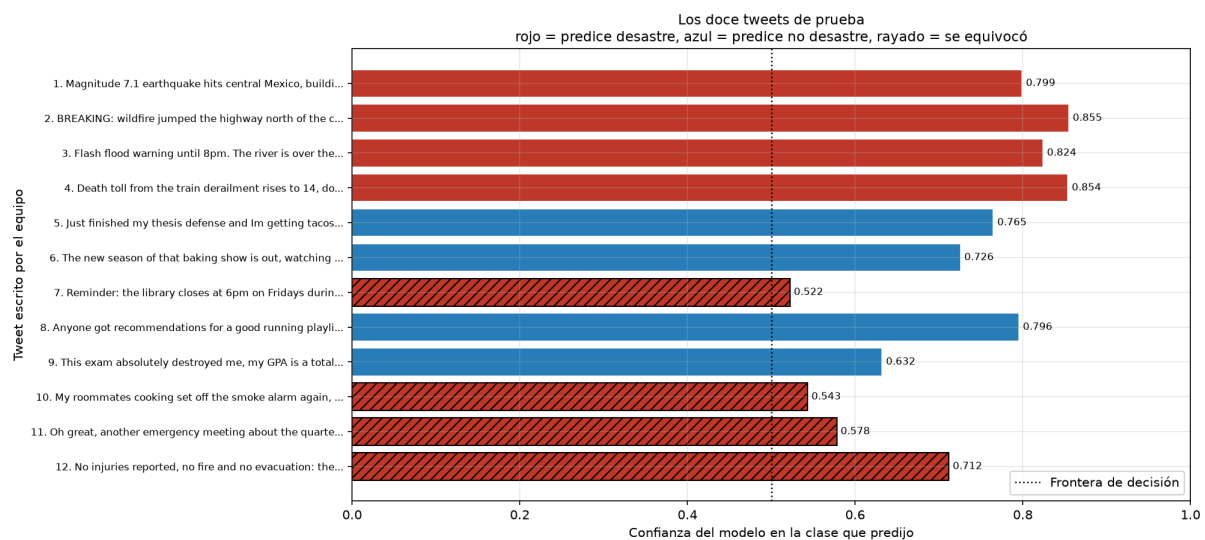


Figura 20. Predicción y confianza de la función sobre los doce tweets de prueba.

La función acierta 8 de 12. Vale la pena notar un detalle antes de entrar a los fallos. La confianza media es 0.781 cuando acierta y 0.589 cuando se equivoca. El modelo no se equivoca con la misma seguridad con la que acierta, y eso hace que la confianza sirva como filtro aunque no esté calibrada.

Por qué falló donde falló

Con un modelo lineal la decisión se puede reconstruir término por término. La contribución de cada uno es su valor TF-IDF multiplicado por su coeficiente, y la suma de todas más el intercepto, que vale -0.405, es lo que decide la clase. No hay que suponer por qué se equivocó, se puede leer.

La Tabla 22 descompone los cuatro fallos.

Tweet	Término	Contribución	Empuja hacia
7	close	+0.233	desastre
7	reminder	+0.180	desastre
7	summer	-0.078	no desastre
10	fire	+0.897	desastre
10	lol	-0.475	no desastre
10	smoke	-0.360	no desastre
11	report	+0.479	desastre
11	emergency	+0.115	desastre
12	fire	+0.565	desastre
12	evacuation	+0.357	desastre
12	no injury	+0.103	desastre

Tabla 22. Descomposición de los cuatro fallos, término por término.

El tweet 12 es el fallo más instructivo de todos. El texto niega un desastre y el modelo lo clasifica como desastre con 0.712 de confianza. Lo empujan fire, evacuation y reported, o sea que las palabras están ahí y la bolsa de palabras no representa que vengan precedidas de un no.

El detalle fino es todavía mejor. El preprocesamiento conserva las negaciones a propósito, así que el no sí llega al modelo e incluso se forma el bigrama no injury. Pero ese bigrama contribuye +0.103 hacia desastre, no en contra, porque los reportes de desastres reales dicen literalmente no injuries reported. Conservar la negación era necesario y no alcanzó. El problema de la negación no se arregla con n-gramas, se arregla con representaciones que modelen el orden de las palabras.

El tweet 10 falla porque fire es la palabra de mayor coeficiente de todo el modelo y aquí aporta +0.897. Es interesante que casi lo salva, porque lol y smoke empujan en contra y la predicción sale con 0.543, prácticamente en la frontera. El tweet 11 falla por sarcasmo, y de forma algo cómica el término que más pesa es report con +0.479, por encima de emergency que aporta +0.115, porque en el corpus report aparece en noticias. El tweet 7 es el falso positivo más inesperado, porque es un tweet cotidiano sobre el horario de una biblioteca, y el culpable es close, que en el corpus aparece en road closed y schools closed.

El tweet 9 sí resistió. Contiene destroyed y disaster y aun así se clasifica bien, lo que muestra que el modelo no funciona por simple coincidencia de palabras clave. Es coherente con los coeficientes, donde varias palabras de catástrofe aparecen del lado de no desastre justo por su uso figurado.

Casos borde

Un tweet vacío, uno que se queda sin contenido después de limpiarlo, uno en otro idioma o un texto más largo que un tweet son entradas reales. Ninguno lanza excepción, y siete de los ocho que se probaron devuelven advertencia. El octavo es un tweet normal de control, que no debe avisar de nada.

La Tabla 23 recoge cómo responde la función en cada caso.

Caso	Predicción	Confianza	Avisa
Cadena vacía	no desastre	0.600	Sí
Valor nulo	no desastre	0.600	Sí
Solo un enlace	desastre	0.865	Sí
Solo menciones y signos	no desastre	0.600	Sí
Solo emoticones	no desastre	0.600	Sí
Texto en español	no desastre	0.542	Sí
Más largo que un tweet	desastre	0.692	Sí

Tweet normal, de control	desastre	0.851	No
--------------------------	----------	-------	----

Tabla 23. Comportamiento de la función ante los casos borde probados.

Dos casos merecen comentario porque no son obvios. El de solo un enlace se clasifica como desastre con 0.865 de confianza, la más alta de la tabla, y sin una sola palabra de contenido. Es la consecuencia directa de la decisión de reemplazar las URL por el token urlweb, que tiene coeficiente +2.26 y es lo único que queda del tweet. La decisión sigue siendo correcta en promedio, pero en este caso extremo produce una predicción muy segura sin ninguna evidencia textual. Por eso la función detecta que el texto limpio solo contiene marcadores y lo advierte.

El texto en español recibe un aviso de cobertura, porque solo 3 de sus 10 palabras están en el vocabulario del modelo. El umbral del aviso, que es 40%, no se eligió a ojo. Se midió la cobertura real de los tweets del conjunto de prueba, cuya mediana es 83%, y solo el 1.87% queda por debajo de 0.40, así que el aviso casi nunca molesta con texto legítimo en inglés.

Análisis de sentimiento

Método y herramientas

El enunciado pide determinar las palabras positivas, negativas o neutras y, con la cantidad de cada una, decidir qué tan positivo o negativo es el tweet. Se implementaron las dos formas y se compararon.

La primera es el conteo directo. Se usa el léxico de VADER, que asigna a cada palabra una valencia entre -4 y +4. Una palabra se cuenta como positiva si su valencia supera +0.5, negativa si baja de -0.5 y neutra en medio. El balance del tweet es la resta entre positivas y negativas.

La segunda es el puntaje compound de VADER, que es la herramienta completa. Además de sumar valencias, ajusta por intensificadores como very, por signos de exclamación, por mayúsculas sostenidas y, lo más importante, por negaciones. VADER fue construido por Hutto y Gilbert justamente para texto de redes sociales, y se usa desde nltk.sentiment.vader. El compound queda entre -1 y +1.

Se eligió el compound como medida principal porque el conteo simple falla de forma sistemática con las negaciones.

La Tabla 24 muestra el problema con tres tweets del corpus donde los dos métodos dan resultados contrarios.

Tweet	Positivas	Negativas	Balance	Compound
No way...I can't eat that shit	0	2	-2	+0.184
People who say it cannot be done should not interrupt	0	1	-1	+0.258
Bioterror lab faced secret sanctions	0	1	-1	+0.206

Tabla 24. Casos donde el conteo simple y el compound de VADER no coinciden.

El umbral para etiquetar el tweet completo se fijó en 0.05, que es el que recomiendan los autores de VADER. Un compound mayor a 0.05 es positivo, menor a -0.05 es negativo, y en medio es neutro. Con ese umbral el corpus queda repartido en 49.25% negativos, 25.42% neutros y 25.33% positivos.

Se probaron umbrales de 0.10, 0.20 y 0.30 y el orden entre clases no cambia, solo crece la franja neutra.

La Figura 21 muestra qué palabras del corpus están en el léxico de VADER y con qué valencia.

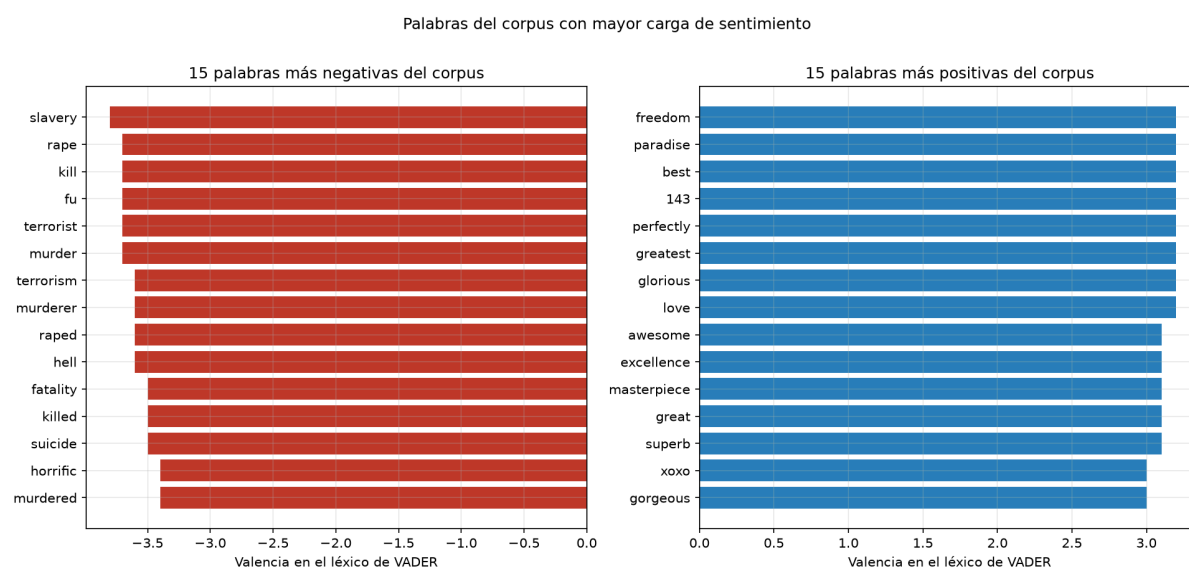


Figura 21. Distribución de valencias del léxico de VADER sobre las palabras del corpus.

Del léxico de VADER, 1,323 palabras distintas aparecen en este corpus. Las más negativas son slavery con -3.8, y después kill, rape, murder y terrorist con -3.7. Las frecuencias importan tanto como las valencias, por ejemplo suicide aparece 114 veces y killed 99.

El preprocesamiento importa, y bastante

Aquí es donde se paga haber hecho dos limpiezas distintas. Se corrió VADER sobre cuatro versiones del mismo texto para medir cuánto cambia el resultado.

La Tabla 25 recoge el resultado sobre las cuatro versiones.

Versión del texto	Compound medio	% negativo	% neutro	% positivo
Texto crudo	-0.1475	48.64	26.44	24.92
Limpieza conservadora	-0.1488	49.23	25.61	25.16
Limpieza agresiva	-0.1550	50.69	23.99	25.32
Versión para VADER	-0.1474	49.25	25.42	25.33

Tabla 25. Efecto de cada versión del preprocesamiento sobre el resultado de VADER.

La Figura 22 compara las cuatro.

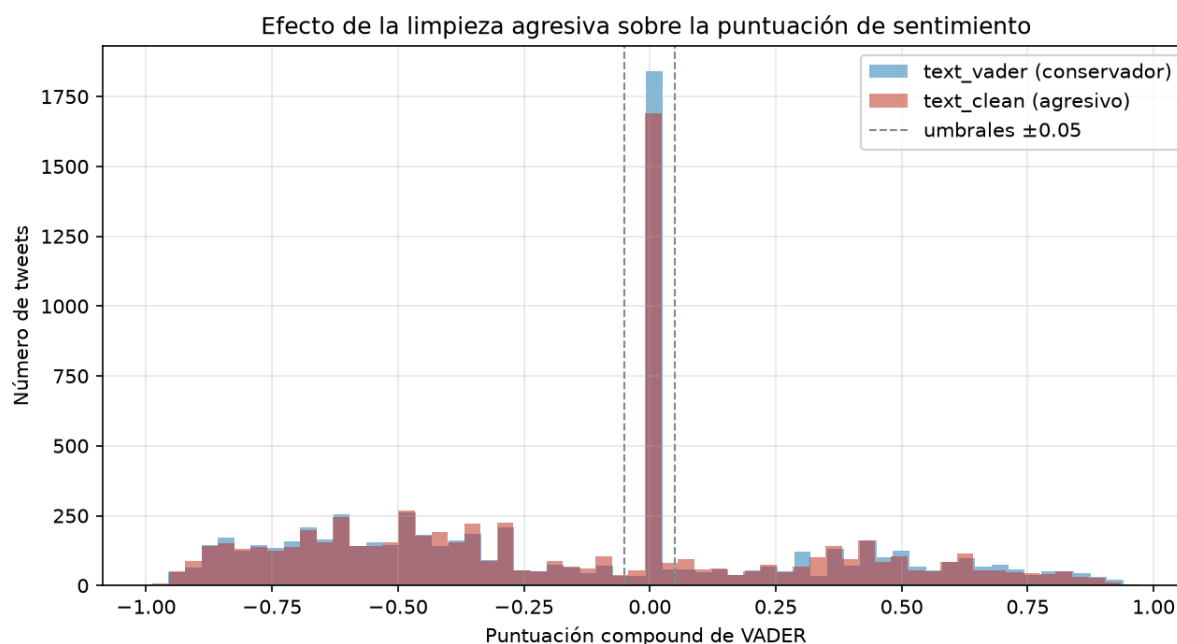


Figura 22. Comparación del resultado de VADER sobre las cuatro versiones del texto.

En el agregado las diferencias parecen chicas, pero en tweets individuales son enormes. Hay casos donde el compound pasa de -0.7218 con la limpieza conservadora a +0.8445 con la agresiva, o sea que la polaridad se invierte por completo. La razón es que la limpieza agresiva quita el no y los signos de exclamación, y VADER se apoya en los dos. Ese es el argumento concreto de por qué el laboratorio mantiene dos pipelines separados.

Los emoticones

El enunciado pregunta si valdrá la pena dejar los emoticones y analizarlos. Se midió en lugar de opinar. En este corpus hay 52 tweets con emoticones de texto, o sea el 0.69%, y no hay ni un solo emoji Unicode.

De esos 52, al traducir el emoticón a un marcador que VADER entiende, 19 cambian de etiqueta de polaridad, que es el 36.54%. Entonces la respuesta tiene dos partes. En este corpus concreto da casi igual, porque 52 tweets sobre 7,485 no mueven ninguna métrica agregada. Pero dentro de los tweets que sí traen emoticón el efecto es grande, y en un corpus más moderno, donde los emojis son la norma, la decisión de conservarlos sería importante. Por eso la limpieza de sentimiento los traduce en lugar de borrarlos.

Contraste con TextBlob

Como comprobación se corrió también TextBlob sobre el mismo corpus. Los dos coinciden en la etiqueta en el 47.12% de los tweets. La discrepancia más grande está en la franja neutra, porque TextBlob marca como neutro todo lo que no encuentra en su léxico, y su léxico es más chico.

La Tabla 26 cruza las etiquetas de los dos métodos.

VADER \ TextBlob	Negativo	Neutro	Positivo
Negativo	1,122	1,777	787
Neutro	160	1,308	435
Positivo	172	631	1,093

Tabla 26. Tabla cruzada entre las etiquetas de VADER y las de TextBlob.

Se prefirió VADER como medida principal por tres razones. Fue construido para texto de redes sociales, maneja negaciones e intensificadores de forma explícita, y devuelve un puntaje continuo que sirve directo para construir la variable de negatividad que pide el enunciado.

Los tweets más negativos y los más positivos

Los diez más negativos

La Tabla 27 trae los diez tweets más negativos con la categoría a la que pertenece cada uno.

Tweet (abreviado)	Compound	Categoría
wreck? wreck wreck wreck wreck wreck...	-0.9883	no desastre
Suicide bomber targets Saudi mosque at least 13 dead	-0.9686	desastre
Suicide bomber kills 15 in Saudi security site mosque	-0.9623	desastre
Nigeria Suicide Bomb Attacks Killed 64 People	-0.9595	desastre
17 killed in Arabia mosque suicide bombing	-0.9552	desastre
at the lake, sees a dead fish, poor little guy	-0.9549	no desastre
illegal alien Charged With Rape and Murder	-0.9538	desastre
Bomb Crash Loot Riot Emergency Pipe Bomb Nuclear	-0.9524	desastre
Bomb head? Explosive decisions dat produced more dead	-0.9500	desastre
Mr. President you are the biggest terrorist	-0.9493	desastre

Tabla 27. Los diez tweets más negativos del corpus según el compound de VADER.

Ocho de los diez están en la categoría de desastre real, lo cual es esperable. Los dos que no lo están son justo los que enseñan algo. El primero repite la palabra wreck trece veces, y VADER suma valencia por cada repetición hasta saturar la escala, aunque el tweet no reporte ningún desastre. El otro es alguien contando que vio un pez muerto en un lago. Los dos son negativos de verdad en tono, y ninguno de los dos habla de una catástrofe.

Los diez más positivos

La Tabla 28 hace lo mismo con los diez más positivos.

Tweet (abreviado)	Compound	Categoría
Want Twister Tickets AND A VIP EXPERIENCE, I would Love	+0.9730	no desastre
I feel like accidents are drawn to you but I'm happy	+0.9564	no desastre
Today's storm will pass, let tomorrow's light greet you	+0.9471	desastre
we enjoyed the show today, the emergency non evacuation	+0.9423	no desastre

Free Ebay Sniping, Lumbar Extender Excellent Condition	+0.9376	no desastre
I think that sounds like a fine plan, little derailment	+0.9356	desastre
I enjoy seeing him body-bagging people, great marketing	+0.9345	no desastre
yeah we survived 9 seasons and 2 movies, hope for the good	+0.9344	no desastre
Super sweet and beautiful	+0.9300	no desastre
45600 ppl got method of meditation, deluge of blessing	+0.9287	no desastre

Tabla 28. Los diez tweets más positivos del corpus según el compound de VADER.

Ocho de los diez están en la categoría de no desastre, que también es lo esperable. Los dos que aparecen marcados como desastre real son claramente errores de anotación, porque uno es una frase motivacional que usa la palabra storm como metáfora y el otro usa derailment en sentido figurado dentro de un intercambio amistoso. Los dos entraron al conjunto porque contienen una keyword de desastre.

Vale la pena notar que en las dos listas aparecen palabras del vocabulario de catástrofe, storm, derailment, body-bagging, accidents. Estar en la lista de los más positivos y contener esas palabras es la misma señal que aparece en los coeficientes del modelo y en el análisis de errores.

¿Son más negativos los tweets de desastre real?

Esta es la pregunta del enunciado y la respuesta es sí, con una condición importante sobre el tamaño de la diferencia.

La Tabla 29 compara la polaridad de las dos categorías.

Categoría	N	Compound medio	Compound mediano	% negativos
No desastre	4,297	-0.0571	0.0000	42.91
Desastre real	3,188	-0.2692	-0.3182	57.78

Tabla 29. Comparación de polaridad entre las dos categorías.

La Figura 23 muestra las dos distribuciones completas.

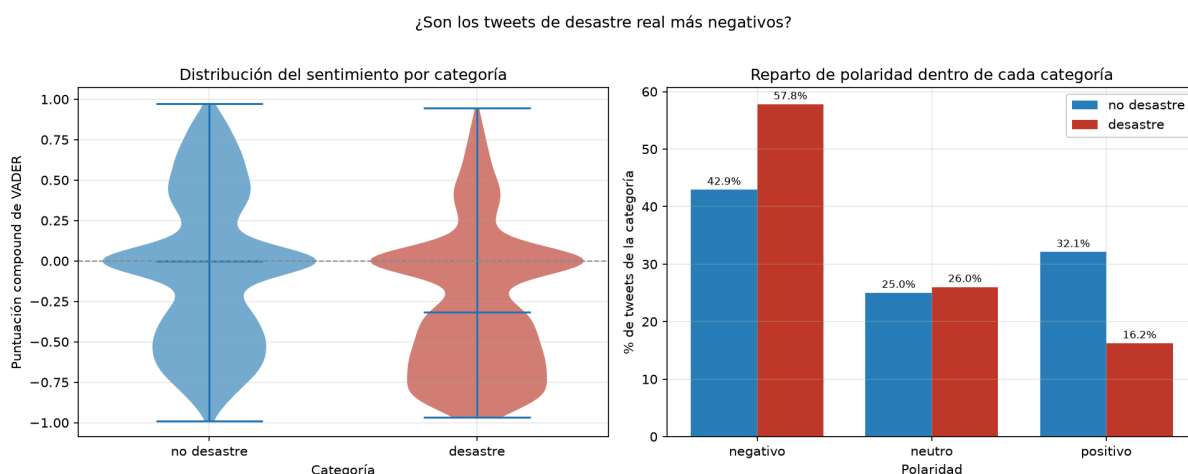


Figura 23. Distribución del compound de VADER en cada categoría.

La mediana de los tweets de desastre es -0.3182 y la de los otros es exactamente 0.0000. La prueba U de Mann-Whitney, que es la adecuada porque las distribuciones no son normales, da un estadístico de 5,088,009 con p menor a 0.001. La diferencia es real y no es casualidad del muestreo.

Ahora bien, el tamaño del efecto cuenta la otra mitad de la historia. La correlación biserial de rangos es 0.2572, que corresponde a un efecto pequeño a moderado, y las dos distribuciones se solapan bastante. El dato que lo explica todo es que el 42.22% de los tweets de desastre real no son negativos, porque muchos son reportes periodísticos factuales o historias de rescate, mientras que el 42.91% de los que no describen ningún desastre sí lo son, porque la gente tuitea sobre su mal día con el vocabulario más dramático que tiene a mano.

Polaridad y factualidad son dos dimensiones distintas. Con eso a la vista se puede anticipar qué va a pasar cuando se agregue la negatividad al modelo, y la siguiente sección lo pone a prueba.

La variable de negatividad y el reentrenamiento

Construcción de la variable

La variable de negatividad se define como (1 menos compound) dividido entre 2. Esa transformación lleva el compound, que va de -1 a +1, a una escala de 0 a 1 donde 0 es lo más positivo y 1 lo más negativo. Se prefirió sobre el componente vader_neg porque el compound ya trae el ajuste por negaciones e intensificadores.

Sobre las 7,485 filas la negatividad tiene media 0.5735. En los tweets de desastre real la media sube a 0.6346 y la mediana a 0.6591, contra 0.5285 y 0.5000 en los otros. La correlación punto-biserial con la clase es 0.2269 con p menor a 0.001, o sea una relación real y moderada.

La Figura 24 muestra cómo se reparte la variable en cada categoría.

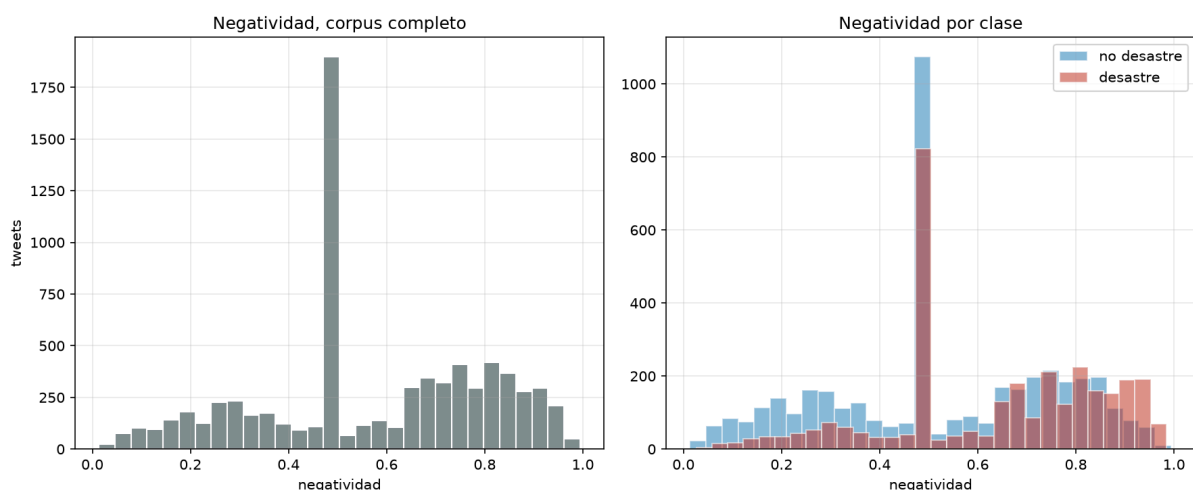


Figura 24. Distribución de la variable de negatividad en las dos categorías.

Resultado del reentrenamiento

Se reentrenó el modelo seleccionado agregando la negatividad como variable numérica. La combinación se hace con un ColumnTransformer, que aplica TF-IDF a la columna de texto y escala la negatividad con MinMaxScaler en paralelo. Se usó MinMaxScaler y no StandardScaler porque este último produce valores negativos, y algunos de los modelos comparados no los aceptan. La partición

es la misma de siempre, leída del disco, así que las cifras son comparables una a una con las del modelo base.

La Tabla 30 pone el modelo base al lado de las dos versiones con información de sentimiento.

Modelo	Exactitud	F1 desastre	Macro-F1	Diferencia
Base, sin sentimiento	0.7923	0.7482	0.7857	0.0000
Con negatividad	0.7822	0.7384	0.7759	-0.0098
Con los 4 componentes de VADER	0.7809	0.7368	0.7746	-0.0111

Tabla 30. Modelo base frente a las dos versiones con información de sentimiento.

La respuesta a la pregunta del enunciado es que no, la inclusión de la variable no mejoró los resultados. Los empeoró en 0.0098 de macro-F1 y en 1.00 puntos de exactitud. Agregar los cuatro componentes de VADER en lugar de solo la negatividad tampoco ayuda, empeora un poco más.

Falta saber si esa caída es real o es ruido de una sola partición. Para eso se aplicó la prueba de McNemar, que compara dos modelos sobre exactamente los mismos tweets y solo mira los casos donde discrepan. De los 1,497 tweets de prueba hay 38 que el modelo base falla y el aumentado acierta, y 53 al revés.

La Tabla 31 trae el resultado de la prueba.

Comparación	Corregidos	Dañados	Estadístico	Valor p
Base contra base más negatividad	38	53	15	0.1418

Tabla 31. Prueba de McNemar entre el modelo base y el modelo con negatividad.

El valor p es 0.1418, muy por encima de 0.05. Entonces la conclusión correcta es que la diferencia no es estadísticamente significativa. El modelo con negatividad no es peor de forma demostrable, y desde luego no es mejor. La variable no aporta.

Por qué no ayudó

El modelo sí le da peso a la variable. Su coeficiente es 1.60, lo que la pone en el mismo orden de magnitud que el término de texto más fuerte, que tiene 3.20. O sea que el modelo la usa, y aun así el resultado empeora.

La explicación está en lo que ya había mostrado el análisis de sentimiento. La correlación entre negatividad y clase es real, pero esa correlación ya está contenida de forma implícita en el texto que procesa el TF-IDF. Las palabras que hacen negativo a un tweet son en buena parte las mismas que ya tienen coeficiente alto en el modelo base, como killed, suicide o bombing. Entonces la variable termina agregando una dimensión más en la que equivocarse cuando el lenguaje figurado suena dramático, sin traer información que el modelo no tuviera ya.

La Tabla 32 trae ejemplos de los dos lados.

Tipo	Tweet (abreviado)	Negatividad
Corregido	raining flooding florida tampabay 18 19 day lost count	0.659
Corregido	kelly osbourne attacked racist donald trump remark	0.895

Dañado	tried 11 eye akame ga kill tokyo ghoul damn bloody	0.953
Dañado	central command confirmed no new casualty please pray	0.182

Tabla 32. Ejemplos de tweets que el reentrenamiento corrigió y de otros que dañó.

Los ejemplos lo muestran bien. Los casos que se corrigen son tweets de desastre con vocabulario muy negativo. Los que se dañan son de dos tipos, un tweet sobre anime con palabras como kill y bloody que ahora se pasa a desastre, y un reporte real que dice que no hay nuevas víctimas, cuya negatividad baja a 0.182 y arrastra la predicción hacia el lado equivocado. Ese segundo caso es exactamente el mismo problema de la negación que apareció en el tweet 12 de la función de clasificación.

Conclusiones

El trabajo produjo un clasificador de tweets de desastre que alcanza 0.7857 de macro-F1 y 79.23% de exactitud sobre 1,497 tweets de prueba, frente a 0.3646 y 57.38% del clasificador trivial. Es TF-IDF con unigramas y bigramas más regresión logística con pesos balanceados, elegido entre catorce combinaciones y cinco variantes de contexto, con el criterio de selección escrito antes de ver los resultados y aplicado sobre validación cruzada del conjunto de entrenamiento.

El hallazgo central del trabajo está en otro lado. Tres análisis independientes llegaron a la misma explicación de qué limita el desempeño.

Desde los datos, el análisis exploratorio encontró que keyword es casi determinista en los extremos y que la incertidumbre se concentra en las keywords de tasa intermedia. Predijo que ahí se concentrarían los errores, y cuando el modelo se entrenó la tasa de error resultó ser 25.03% en esas keywords contra 10.34% en las de tasa extrema alta.

Desde el modelo, los coeficientes contienen palabras de catástrofe del lado de no desastre, como ruin, wrecked, traumatised y avalanche. El modelo aprendió sin que nadie se lo dijera que ese vocabulario es marcador de uso figurado, y el análisis de errores mostró que cinco de cada diez falsos positivos son justamente eso.

Desde el sentimiento, los tweets de desastre real son medibles más negativos, pero con efecto pequeño a moderado y distribuciones muy solapadas, porque el 42.22% de los de desastre real no son negativos. Esa tercera línea permitió anticipar el resultado del reentrenamiento antes de ejecutarlo, y el experimento lo confirmó.

Hay entonces un techo en este conjunto que no depende del modelo. El archivo crudo trae 18 textos idénticos etiquetados a la vez como desastre y como no desastre, o sea que ni los anotadores coincidieron entre ellos. La tipificación de errores lo confirma desde el otro lado, con ocho de diez falsos negativos donde la anotación del conjunto es la que cuesta defender. Una parte del error medido es ruido de anotación, y eso explica por qué catorce combinaciones caben en 0.035 de macro-F1.

Conviene decir también lo que no funcionó, porque cada cosa enseña algo. La keyword como variable del modelo resta 0.0187 porque ya está escrita dentro del texto en el 94.10% de los casos. La variable de negatividad resta 0.0098 sin significancia estadística. Los bigramas suman 0.0001 y los trigramas 0.0036, con casi el doble y el triple de vocabulario. Random Forest queda por debajo de los otros tres algoritmos y es el más caro.

Y una cosa que sí funcionó yendo contra la recomendación habitual, que fue no borrar las URL ni los números. La presencia de una URL aparece en el 67.35% de los tweets de desastre contra el 41.73%

del resto, y los dígitos en el 72.74% contra el 49.90%. El token urlweb terminó siendo el noveno término de mayor coeficiente del modelo.

El límite conceptual del enfoque quedó documentado con un caso concreto. El tweet que dice no injuries reported, no fire and no evacuation niega un desastre y el modelo lo clasifica como desastre con 0.712 de confianza, porque el bigrama no injury empuja hacia desastre. Conservar la negación era necesario y no fue suficiente. Es una limitación del enfoque de bolsa de palabras, y ningún ajuste de parámetros la resuelve.

Referencias

Bird, S., Klein, E., y Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media. Módulo NLTK, usado para palabras vacías, tokenización, lematización con WordNetLemmatizer y el analizador de sentimiento VADER.

Hutto, C. J., y Gilbert, E. (2014). VADER, a Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. Eighth International Conference on Weblogs and Social Media (ICWSM-14). Ann Arbor, Michigan.

Jurafsky, D., y Martin, J. H. (2008). Speech and Language Processing. Capítulo de N-gramas. Recuperado de <https://web.stanford.edu/~jurafsky/slp3/>

Kaggle. Natural Language Processing with Disaster Tweets. Recuperado de <https://www.kaggle.com/competitions/nlp-getting-started/data>. Los tweets fueron recolectados por Figure Eight y distribuidos en su colección Data For Everyone.

Loria, S. (2018). TextBlob, Simplified Text Processing. Usado como contraste del análisis de polaridad de VADER.

Pedregosa, F., y otros (2011). Scikit-learn, Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830. Módulos usados, CountVectorizer, TfidfVectorizer, Pipeline, ColumnTransformer, LogisticRegression, LinearSVC, ComplementNB, RandomForestClassifier, StratifiedKfold y las métricas de clasificación.

Mueller, A. (2020). WordCloud for Python. Usado para las nubes de palabras de la sección exploratoria.

Virtanen, P., y otros (2020). SciPy 1.0, Fundamental Algorithms for Scientific Computing in Python. Nature Methods, 17, 261-272. Usado para la prueba U de Mann-Whitney y la prueba de McNemar.